
Learning Analytics with OULAD

From Relational Data to Validated SQL

Course Edition 0.1

OULAD, Relational Data, PostgreSQL, Data Validation, and SQL Foundations

A project-based course built from the real
Open University Learning Analytics Dataset project

Jose G. Chavez

July 2026

Edition statement

Edition status

This is a **substantive first edition**, not the complete finished course. Chapters 1–9 fully teach the foundations requested for this release. Later chapters appear in the table of contents as clearly marked forthcoming architecture. Their topics are not described as implemented lessons.

This educational artifact documents the existing Learning Analytics project. It does not alter the PostgreSQL schemas, SQL pipeline, Python package, model, saved analytical results, raw data, or dashboard. Verified numerical results come from versioned project outputs. New instructional queries are labeled as instructional examples rather than production pipeline code.

What this edition teaches

By the end of Edition 0.1, a learner should be able to:

- explain the purpose and limits of learning analytics;
- describe every OULAD source table and its observation grain;
- reason about keys, relationships, nulls, and join risks;
- prepare a PostgreSQL 16 environment using the project-supported paths;
- download, verify, and import the official OULAD archive;
- run and interpret source, key, relationship, range, and category checks;
- write introductory PostgreSQL queries against real OULAD tables; and
- validate query outputs instead of accepting plausible-looking results.

Evidence standard

The course uses three labels:

Verified result A value present in a saved project output or executable validation result.

Instructional query New documentation-only SQL written to teach a concept using the project's real schema.

Conceptual extension An educational idea that is useful but is not part of the implemented project pipeline.

Contents

Edition statement	i
I Foundations	1
1 Course Orientation	2
1.1 Start here: the basic vocabulary	2
1.2 The project you are learning through	4
1.2.1 Technical names for the workflow steps	4
1.3 Edition 0.1 boundary	5
1.4 Who this course is for	5
1.4.1 Prerequisites	6
1.5 How to use each chapter	6
1.6 A map of the repository	6
1.7 Evidence and honesty	7
1.8 Knowledge check	7
2 Learning Analytics and the OULAD Problem	9
2.1 Vocabulary for asking questions with data	9
2.2 What learning analytics means	10
2.3 Four concepts that must not be collapsed	11
2.4 Description, prediction, and causation	11
2.5 The implemented project questions	12
2.6 Potential value	12
2.7 Risks and boundaries	13
2.8 Operational definitions	13
2.9 Statistical reasoning preview	14
2.10 Knowledge check	14

3	The OULAD Source Tables and What Each Row Means	16
3.1	Read one CSV row before reading seven tables	16
3.2	Where the data came from: provenance	17
3.3	OULAD vocabulary	17
3.4	The seven source files	18
3.5	A relational map	18
3.6	Grain before keys	18
3.7	Time is relative	20
3.8	Missing values and absent rows	20
3.9	Representational limitations	20
3.10	Knowledge check	21
4	Relational Database Foundations	23
4.1	The relational model	23
4.2	Why the project has layers	24
4.3	Keys	24
4.3.1	Why the key is composite	25
4.4	Relationships and cardinality	25
4.4.1	The five join questions	25
4.5	Referential integrity	26
4.6	Nulls and constraints	26
4.7	Normalization and denormalization	26
4.8	Views, materialized views, and indexes	27
4.9	Knowledge check	27
II	Build the Validated Database	29
5	PostgreSQL and Project Setup	30
5.1	What the environment must provide	30
5.2	Start from the repository root	31
5.3	Create the Python environment	31
5.3.1	What the commands do	31
5.4	Understand the database URL	32
5.5	Path A: Docker Compose	32
5.6	Path B: Trusted local PostgreSQL	32
5.7	Environment files versus exported variables	33

5.8	Create the project schemas	33
5.9	Full setup checkpoint	33
5.10	Knowledge check	33
6	Downloading, Auditing, and Importing Raw OULAD Data	36
6.1	The raw-data contract	36
6.2	Download and source audit	37
6.3	Why raw tables preserve source shape	37
6.4	Import order	38
6.4.1	Why \copy is used	38
6.5	Failure behavior	39
6.6	Validate the imported row counts	39
6.7	From raw text to staging types	40
6.8	Knowledge check	40
7	Data Validation as an Executable Contract	42
7.1	Validation is not decoration	42
7.2	Severity	43
7.3	Source-count checks	43
7.4	Natural-key checks	43
7.5	Relationship checks with anti-joins	44
7.5.1	Reasoning through the query	44
7.6	Range and category checks	45
7.7	Date and temporal checks	45
7.8	Known profiles are evidence	45
7.9	Run the quality gate	45
7.10	How to investigate a failed check	46
7.11	Knowledge check	47
III	Query the Evidence	49
8	SQL Foundations with Real OULAD Questions	50
8.1	The shape of a query	50
8.2	Inspect a table safely	50
8.2.1	Question	51
8.2.2	Reasoning	51

8.3	Count rows and distinct entities	51
8.3.1	Question	51
8.3.2	Why the two counts differ	52
8.4	Filter rows	52
8.4.1	Comparisons and logical operators	52
8.4.2	BETWEEN and LIKE	52
8.5	Work with null values	52
8.5.1	Question	52
8.6	Create an interpretable category with CASE	53
8.6.1	Question	53
8.7	Aggregate outcomes	53
8.7.1	Question	53
8.8	Use numerical aggregates	54
8.8.1	Question	54
8.9	Group by module-presentation	54
8.9.1	Question	54
8.10	Conditional aggregation	55
8.10.1	Question	55
8.11	Validation habits for introductory SQL	55
8.12	Knowledge check	55
9	Cumulative Lab: From Source Files to Defensible Findings	58
9.1	Learning objectives	58
9.2	Scenario and deliverables	58
9.3	Part A: Write the source and grain map	59
9.4	Part B: Record the environment	59
9.5	Part C: Audit and import	60
9.5.1	Reconciliation table	61
9.6	Part D: Interpret the quality gate	61
9.7	Part E: Build the six-query portfolio	62
9.7.1	Query 1: confirm the attempt population	62
9.7.2	Query 2: describe final outcomes	62
9.7.3	Query 3: count repeat appearances	62
9.7.4	Query 4: inspect withdrawal timing	63
9.7.5	Query 5: profile assessment due-date missingness	63

9.7.6 Query 6: bound engagement timing	63
9.8 The evidence log	63
9.9 Common mistakes	64
9.10 Transfer task	64
9.11 Guided solutions	64
9.12 Chapter summary	64
 IV Forthcoming Course Architecture	65
 10 Forthcoming Course Modules	66
 11 SQL Joins and Analytical Grain	67
 12 Intermediate and Advanced SQL	68
 13 Python and pandas	69
 14 Exploratory Data Analysis	70
 15 Statistics	71
 16 Visualization	72
 17 Engagement Feature Engineering	73
 18 Assessment Feature Engineering	74
 19 Time-aware Analytical Design	75
 20 Leakage Prevention	76
 21 Predictive Modeling	77
 22 Model Evaluation	78
 23 Error Analysis and Interpretation	79
 24 Responsible Educational Analytics	80
 25 Reproducibility and Software Quality	81

26 Final Capstone	82
26.1 What the roadmap does not imply	82
V Reference Material	83
A Guided Solutions	84
A.1 Course orientation	84
A.1.1 Solution to the orientation exercise	84
A.2 Learning analytics and the problem	84
A.2.1 Solution to the classification check	84
A.2.2 Solution to the question-design exercise	84
A.3 The OULAD data	85
A.3.1 Solution to the source-grain map	85
A.3.2 Solution to the attempt trace	85
A.4 Relational foundations	85
A.4.1 Solution to the dangerous join	85
A.4.2 Solution to the database-rules exercise	86
A.5 PostgreSQL setup	86
A.5.1 Solution to the environment exercise	86
A.6 Raw import	86
A.6.1 Solution to the reconciliation exercise	86
A.6.2 Solution to the failed-cast exercise	86
A.7 Data validation	87
A.7.1 Solution to the validation specification	87
A.7.2 Solution to the orphan-debugging exercise	87
A.8 Introductory SQL	87
A.8.1 Solution to the inspection exercise	87
A.8.2 Solution to the outcome exercise	88
A.8.3 Solution to the missingness exercise	88
A.8.4 Solution to the repeat-appearance exercise	88
A.9 Cumulative lab	88
A.9.1 Solution to lab checkpoint 1	88
A.9.2 Solution to lab checkpoint 2	89
A.9.3 Solution to lab checkpoint 3	89
A.9.4 Solution to lab checkpoint 4	89

A.9.5 Solution to lab checkpoint 5	89
A.10 How to use these solutions	90
B Glossary	91
C References	95
Build and provenance note	97

List of Figures

1.1	The implemented project lineage. Edition 0.1 teaches the workflow through the SQL foundations stage; later stages are forthcoming course chapters.	4
2.1	Recorded final outcomes across 32,593 student-module attempts. This verified chart describes the dataset; it does not explain the outcomes.	12
3.1	Conceptual relationships among the seven source tables. Arrows show how identifiers can connect tables. The original CSV files do not enforce these matches as database rules.	19

List of Tables

1.1	First vocabulary for the course	2
1.2	Course Edition 0.1 scope	5
1.3	Prerequisite expectations and support	6
1.4	Project paths used in Edition 0.1	7
2.1	Basic analytical vocabulary	9
2.2	Three kinds of analytical claim	11
3.1	Verified OULAD source files used by the project	18
3.2	Source-grain reasoning	19
4.1	Database layers and their contracts	24
5.1	Environment checkpoint	34
7.1	Implemented quality severity contract	43
7.2	Selected persisted quality profiles	46
8.1	Verified recorded outcome counts	54
8.2	SQL reasoning template	56
9.1	Cumulative lab source and grain map	59
9.2	Import reconciliation record	61
9.3	Evidence-log template	64
A.1	Suggested grains and candidate keys	85
B.1	Course glossary	91

Part I

Foundations

Course Orientation

Learning objectives

After completing this chapter, you will be able to:

- state what you will build in Course Edition 0.1;
- distinguish project implementation from course documentation;
- identify the prerequisites, checkpoints, and evidence standards;
- describe the complete project workflow at a high level; and
- use the course’s read-run-validate-reflect study cycle.

1.1 Start here: the basic vocabulary

In plain language

You do not need to arrive knowing database or data-science language. At the simplest level, this project is a folder containing two kinds of files: data files that hold recorded information and instruction files that tell the computer how to check, store, summarize, and display that information.

The terms in [table 1.1](#) will appear throughout the course. Read them once now and return to the table whenever a word feels unfamiliar. Later chapters will make the database terms more precise.

Table 1.1: First vocabulary for the course

Term	Meaning in this course
Data	Recorded facts or values. A dataset is an organized collection of those records. Data can be useful without being complete or perfectly accurate.
Data science	The combined practice of using programming, databases, statistics, and subject knowledge to learn from data and communicate what the evidence supports.

Term	Meaning in this course
File, folder, path	A file stores content. A folder, also called a directory, groups files. A path is the address of a file or folder, such as <code>sql/02_staging.sql</code> .
CSV file	A plain-text table in which values are separated by commas. CSV means comma-separated values. Chapter 3 opens one and explains every part.
Archive	One packaged file that contains other files, commonly in ZIP format. Projects often download one archive and then unpack its contents.
Table, row, column	A table arranges data in a grid. A row usually describes one recorded thing or event. A column contains one kind of value, such as a student identifier or date.
Code and script	Code is text written as instructions for a computer. A script is a file of code intended to perform a sequence of steps.
Terminal and command	A terminal is a text-based way to interact with the computer. A command is one instruction typed there, such as asking PostgreSQL to run a SQL file.
Database	Software-managed storage for organized data. Unlike a single spreadsheet, a database can hold many related tables and enforce rules about them.
PostgreSQL	The database program used by this project. It stores the OULAD tables and runs the project's SQL instructions.
SQL and query	SQL is a language for working with relational databases. A query is a SQL request, often asking PostgreSQL to return, combine, summarize, or check rows.
Python	A general-purpose programming language. This project uses Python to download and audit files and, in later stages, to analyze, chart, and model data.
Repository	The main project folder and its organized contents. The word often implies that file history is tracked with Git, a file-history tool, but you do not need Git to read the course or query the database.
Implementation and documentation	The implementation is the working code and settings that perform the project. Documentation explains what that implementation does and how to use or evaluate it.
Input and output	An input is what a step receives. An output is what it produces. The official CSV files are inputs; checked tables and charts are outputs.
Workflow or pipeline	An ordered sequence in which the output of one step becomes the input to the next. <i>Pipeline</i> is the common technical word; <i>workflow</i> is the same idea in plainer language.
Analysis, model, dashboard	Analysis examines data to answer a question. A model is a mathematical rule learned or estimated from data. A dashboard is a visual display of selected results. They are different products and should not be confused.
Reproducible	A result is reproducible when another person can follow the recorded source, code, settings, and steps and obtain the same result.

Validation checkpoint

Before continuing, make sure you can explain this sentence:

“A script reads an input CSV file, loads its rows into PostgreSQL, and a SQL query produces an output table.”

It is fine to return to the vocabulary table while translating it into your own words.

1.2 The project you are learning through

This course is built around a real, implemented data-science project: *Learning Analytics: Engagement, Assessment, and Outcome Forecasting*. The project uses the Open University Learning Analytics Dataset (OULAD), a set of seven linked CSV files published for research and education [Kuzilek et al., 2015, 2017].

The implemented workflow is:

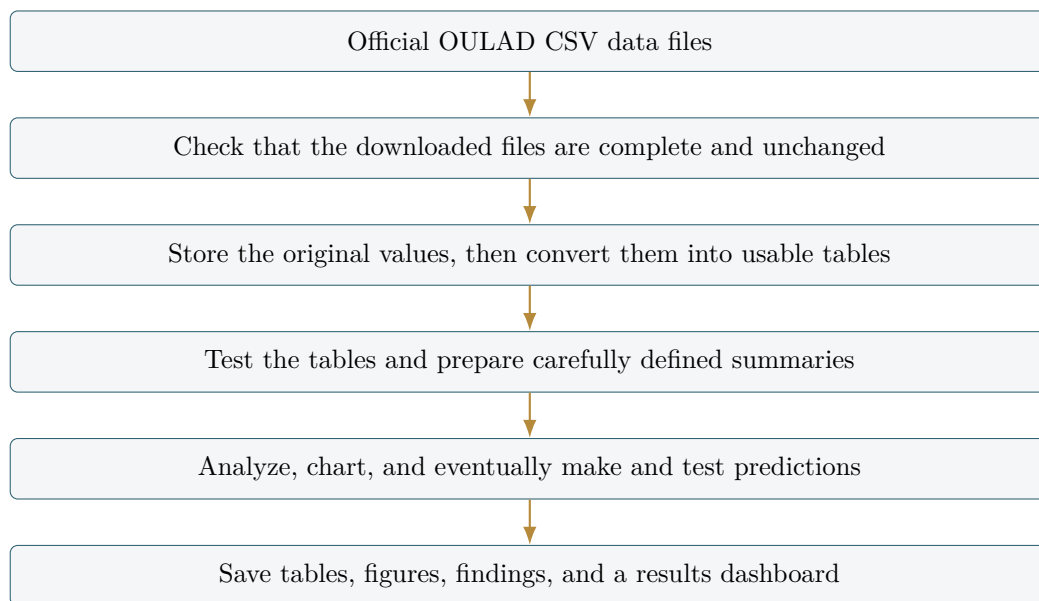


Figure 1.1: The implemented project lineage. Edition 0.1 teaches the workflow through the SQL foundations stage; later stages are forthcoming course chapters.

1.2.1 Technical names for the workflow steps

The diagram begins in everyday language. The project files use the following shorter technical names:

Checksum A compact fingerprint used to check whether a file’s bytes match the expected file.

Audit A recorded inspection of source files, such as checking names, row counts, columns, and missing values.

Raw layer Database tables that preserve source text as closely as practical.

Staging layer An intermediate area where text is converted into meaningful types, names, and missing values.

Core layer The main checked tables used for analysis.

Data validation Tests that compare the data with explicit expectations.

Snapshot A saved view of information as of a particular course week.

Lineage The traceable path from original input to a reported result.

In plain language

You are not starting with a small practice database. You are learning to understand, reproduce, and question an existing analytical system. The code files are the case under study. The course provides the explanation, practice, and checking habits that code alone does not provide.

1.3 Edition 0.1 boundary

The first edition fully develops the foundation shown in [table 1.2](#). It deliberately does not rush through the modeling half of the project. Those chapters need their own worked examples, exercises, and careful treatment of temporal leakage, which means accidentally using information from the future to make an earlier prediction.

Table 1.2: Course Edition 0.1 scope

Area	Status in this edition	Concrete learner output
Learning analytics and OULAD	Complete	Careful question statement and source-table map
Relational foundations	Complete	Row meaning (grain), key, and relationship worksheet
PostgreSQL setup	Complete	Verified database connection
Raw import	Complete	Seven populated raw tables
Data validation	Complete	Interpreted quality-check report
Introductory SQL	Complete	Reproducible OULAD query set
Joins through modeling	Forthcoming	Architecture only; no claim of completion

1.4 Who this course is for

The intended learner wants to build applied SQL and data-science skill through one coherent project. You do *not* need prior OULAD knowledge or advanced SQL. The course teaches the relational and SQL ideas required by this edition.

1.4.1 Prerequisites

Table 1.3: Prerequisite expectations and support

Skill	Expected starting point	Support in this edition
Files and folders	Able to locate a project folder	Exact paths from the main project folder are supplied
Terminal	Able to run a copied command	Commands are explained before use
Python	No pandas experience required yet	Installation is covered; Python analysis is forthcoming
PostgreSQL	No prior database administration required	Two supported setup paths are taught
SQL	No prior SQL required	Starts with SELECT , not advanced windows
Statistics	Basic percentages are helpful	Formal statistical reasoning is forthcoming
Git	Useful but not required for the lessons	No Git operation is required to query the database

1.5 How to use each chapter

Use the same five-step cycle throughout the course:

1. **Read the question.** Begin with the analytical reason for a command or query.
2. **Predict the output.** State the expected row grain, columns, and possible missing values before running code. The **grain** is what one output row will represent.
3. **Run the smallest useful step.** Do not execute the entire workflow merely to learn one SQL clause, or instruction.
4. **Validate.** Compare counts, keys, ranges, and categories with a known contract.
5. **Reflect and transfer.** Explain what would change with a different dataset.

Connection to the OULAD project

The project itself follows this logic. For example, `sql/04_quality.sql` does not merely calculate summaries. It records the expected rules, called a data contract, counts rows that break them, records how serious each failure is, and preserves the result for later inspection.

1.6 A map of the repository

Table 1.4: Project paths used in Edition 0.1

Path	Role in the learning sequence
README.md	Project scope, architecture, results, and reproduction order
DATA_LICENSE.md	Dataset citation, license, and access note
src/learning_analytics/download.py	Download and file-fingerprint verification
src/learning_analytics/audit.py	Source profiling and saved audit outputs
sql/00_database.sql	Six named database areas, called schemas
sql/01_raw_tables.sql	Source-faithful landing tables
sql/02_staging.sql	Types, names, and missing-value normalization
sql/03_core.sql	Keys, constraints, and core relational tables
sql/04_quality.sql	Executable data-quality framework
sql/gallery/queries.sql	Query sequence used as the SQL source bank
reports/tables/	Verified source counts and quality results

1.7 Evidence and honesty

A polished number is not automatically trustworthy. In this course, a numeric claim must be connected to a saved output or an executable query. For example, the statement “OULAD contains 10,655,280 VLE activity records” is supported by `reports/tables/source_file_profile.csv` and an error-level SQL row count check. By contrast, a query whose output has not been saved is described by its expected columns and row meaning, sometimes called its *shape*, not assigned an invented result.

1.8 Knowledge check

1. Which stages of [figure 1.1](#) are fully taught in Edition 0.1?
2. What is the difference between a verified result and an instructional query?
3. Why does the course ask you to predict output grain before running SQL?
4. Which skills are legitimate prerequisites, and which are taught here?

Exercise 1.1: Write your study contract

In four short paragraphs, record:

1. what you expect to build by the end of Edition 0.1;
2. which prerequisite is currently weakest;
3. how you will validate a result before trusting it; and
4. which forthcoming topic you most want to reach.

Your output should be a one-page study note. No code is required. See the guided solution in

appendix A.1.1.

Common mistakes and troubleshooting

- **Treating the dashboard as the course.** It communicates results; it does not teach the implementation.
- **Running everything immediately.** Large imports obscure basic concepts. Follow the chapter checkpoints.
- **Equating completion with understanding.** A command that exits successfully can still produce rows at the wrong level of detail.

Transfer to another dataset

Before working with any unfamiliar project folder, draw the path from source to output, distinguish generated files from editable source, and decide which claims can be regenerated. This habit transfers to nearly every analytical project.

Chapter summary

Edition 0.1 is a bounded, project-based course release. It teaches the OULAD context, the organization of linked database tables, PostgreSQL setup, source import, validation, and SQL foundations. It does not pretend that the later analytical and modeling chapters are finished. Each lesson follows a read, predict, run, validate, and reflect cycle.

Learning Analytics and the OULAD Problem

Learning objectives

After completing this chapter, you will be able to:

- define learning analytics, engagement, assessment performance, and outcome forecasting in plain and technical language;
- distinguish descriptive, predictive, and causal questions;
- explain the benefits and risks of using educational records;
- state what the implemented OULAD project can and cannot establish; and
- write a responsible analytical question with an explicit unit and claim boundary.

2.1 Vocabulary for asking questions with data

Chapter 1 introduced the software words. This chapter introduces the words used to turn a real-world concern into a question that data can answer.

Table 2.1: Basic analytical vocabulary

Term	Meaning in this course
Record or observation	One recorded case. Depending on the table, it might be a course offering, a student's course attempt, an assessment, or one day's online activity.
Variable	One recorded characteristic that can differ across observations, such as final result, score, or number of clicks. In a table, a variable is usually stored in a column.
Measure or metric	A value chosen to summarize something, such as attempt count or average score. A metric is a definition made by an analyst; it is not the complete reality behind the number.

Term	Meaning in this course
Outcome	The event or result being described or predicted. In one OULAD question it might be the final-result category; in another it might be whether the next assessment is missing.
Population	The full set of observations a claim is about. “All attempts recorded in OULAD” is a population; “all university students everywhere” is not the population represented by this dataset.
Unit of observation	What one record represents. This is also called the observation grain. A student and a student-course attempt are different units.
Proportion and percentage	A proportion is a part divided by a whole, such as withdrawn attempts divided by all attempts. Multiplying a proportion by 100 expresses it as a percentage. The denominator, or whole, must always be named.
Pattern or association	A difference or relationship observed in records. An association can be real without showing that one variable caused the other.
Proxy	A recorded stand-in for something that is difficult to measure directly. Clicks can be a proxy for one aspect of online activity, but they are not a direct measure of attention, effort, or learning.
Prediction or forecast	An estimate of a later or unknown recorded outcome. A prediction says what the model expects, not why the outcome will occur.
Model	A mathematical rule that uses input variables to calculate a summary or prediction. Models can be useful and still be uncertain or wrong.
Claim boundary	A sentence stating how far the evidence reaches. It names what a result describes and what it does not prove.

In plain language

An analytical question should identify the people or records under discussion, what one record represents, what is measured, and how strong an answer the data can support. Those four choices prevent a number from quietly becoming a much larger claim.

2.2 What learning analytics means

Learning analytics is the organized use of data about learners and learning environments to understand and improve educational processes. The phrase covers many tasks: describing participation, finding data-quality problems, evaluating assessment patterns, forecasting a recorded event, or supporting a conversation about course design.

In plain language

Learning analytics asks what can be learned from records created while education is taking place. The records are traces of a system, not complete measurements of a person.

The OULAD project studies records from selected Open University module presentations, meaning particular deliveries of a course, in 2013 and 2014 [Kuzilek et al., 2015, 2017]. Its source tables connect module offerings, registrations, assessments, recorded student information, online resources,

submissions, and Virtual Learning Environment (VLE) interactions. A VLE is the institution's online course system.

2.3 Four concepts that must not be collapsed

Student engagement A broad educational idea concerning participation and involvement. In this project, it is represented only by limited indicators such as recorded VLE clicks and active days.

Assessment performance Recorded evidence associated with coursework or examinations, including submission timing, missing submissions, scores, and weights.

Course outcome OULAD's recorded final category: Distinction, Pass, Fail, or Withdrawn.

Outcome forecast A model estimate about a later recorded event using information available at a specified earlier point.

The first three concepts describe different objects. A student can click often without understanding the material; a low click count can coexist with offline study; a missing submission can reflect circumstances not captured in the data; withdrawal is an administrative outcome, not a diagnosis.

2.4 Description, prediction, and causation

Table 2.2: Three kinds of analytical claim

Claim type	Example OULAD question	What it can establish	What it cannot establish
Descriptive	How many attempts ended in withdrawal?	A count or proportion in the observed records	Why withdrawal occurred
Predictive	Can information available this week rank later assessment risk?	How well a forecast works when checked on records not used to build it	Causal explanation or individual need
Causal	Would a new support program reduce withdrawal?	The change caused by the program, if the study fairly compares what would happen with and without it	Not available from these existing records alone

Connection to the OULAD project

The project's saved outcome table records 3,024 Distinction, 12,361 Pass, 7,052 Fail, and 10,156 Withdrawn student-module attempts. The withdrawal proportion is 31.2 percent. This is a descriptive fact about recorded attempts. It does not identify a cause.

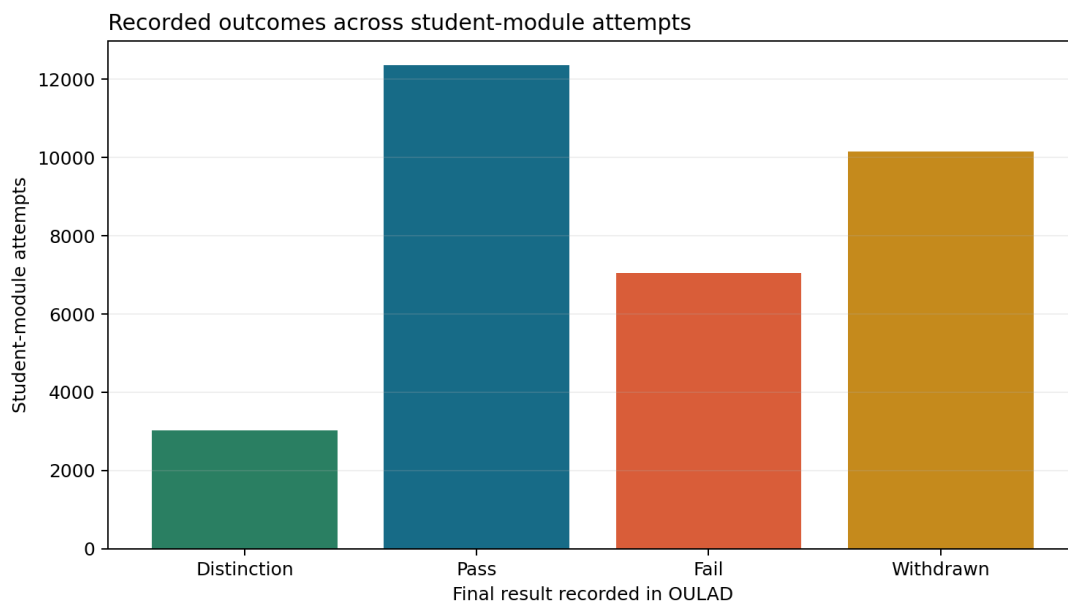


Figure 2.1: Recorded final outcomes across 32,593 student-module attempts. This verified chart describes the dataset; it does not explain the outcomes.

2.5 The implemented project questions

The project moves from description toward prediction through five questions:

1. How do engagement volume, consistency, and inactivity differ across weeks and module presentations?
2. How do assessment availability, submission timing, missing submissions, and weighted progress, where assessments contribute according to their course weights, vary?
3. What activity patterns appear before recorded withdrawal?
4. At the end of a course week, how well can available information forecast a missing or below-pass next assessment?
5. Do predicted percentages agree with observed percentages (**calibration**)? How many cases are flagged at a chosen cutoff (**threshold workload**)? Does performance change by course week or on a presentation excluded while building the model?

Only the first part of this progression is taught in Edition 0.1. The complete questions are introduced now so that database design is connected to later analytical purpose.

2.6 Potential value

Responsible educational analytics can:

- reveal where records are incomplete or inconsistent;
- compare patterns across courses without assuming they have one cause;

- make operational definitions explicit;
- support investigation of course design and assessment timing;
- expose uncertainty and tradeoffs before a decision; and
- help institutions ask better questions about support.

These benefits depend on careful data definitions. A count at the wrong grain can produce a convincing but false conclusion.

2.7 Risks and boundaries

Educational data concern people in settings with unequal resources and power. Important risks include revealing private information; measuring some students or behaviors less accurately than others; repeating inequalities already present in historical records; attaching harmful labels; allowing a prediction to shape the very outcome it predicts; and using analysis to punish rather than support.

Validation checkpoint

Before presenting an educational result, answer:

1. What exactly was measured?
2. At what grain was it measured?
3. What important behavior is absent from the data?
4. Is the claim descriptive, predictive, or causal?
5. What harmful interpretation could a reader make?
6. What human review and opportunity to question or appeal the result would be required before action?

A “high-risk” prediction must never be treated as a judgment about ability, effort, motivation, or character. In the implemented project, the primary model predicts a later recorded assessment event, not a student’s worth or instructional diagnosis. The secondary withdrawal investigation is explicitly documented as unsuitable for individual use.

2.8 Operational definitions

An **operational definition** states how an abstract idea becomes a measurable variable. For example:

- “weekly activity” becomes the sum of `sum_click` within a course-relative week;
- “active days” becomes the number of distinct recorded activity days;
- “missing assessment” becomes an eligible student-assessment pair with no matching submission row;
- “withdrawal” becomes a recorded final result and, where present, an unregistration day.

An operational definition makes analysis reproducible, but it does not erase the difference between the variable and the underlying educational concept.

2.9 Statistical reasoning preview

The dataset is not a random sample in which members of a larger population had a known chance of selection. It covers selected, anonymized modules from one institution and period. Therefore:

- a large row count does not guarantee that a result applies elsewhere;
- differences between modules may reflect design, population, policy, or measurement;
- repeated records from the same student are not separate, unrelated people;
- associations do not establish causes; and
- model performance on later presentations does not establish performance at another institution.

Formal statistical and modeling treatment is forthcoming. Edition 0.1 uses these ideas to discipline SQL interpretation.

2.10 Knowledge check

1. Why is a VLE click not a direct measure of learning?
2. Classify each project question as descriptive, predictive, or causal.
3. What does the recorded withdrawal proportion establish?
4. Give one potential benefit and two potential harms of educational analytics.
5. What is an operational definition?

Exercise 2.1: Repair three overclaims

Rewrite each claim so that it matches the available evidence:

1. “Students with fewer clicks were less motivated.”
2. “Missing work caused withdrawal.”
3. “The model identifies students who will fail.”

For each revision, name the measured variable, unit of observation, claim type, and one missing source of context. See the guided solution in [appendix A.2.1](#).

Exercise 2.2: Frame an OULAD question

Write one descriptive OULAD question. It must specify:

- the population;
- the observation grain;
- the variable or event;
- the grouping or comparison; and
- a statement of what the result would not prove.

See the guided solution in [appendix A.2.2](#).

Common mistakes and troubleshooting

- Using “student” when the row actually represents a student-module-presentation attempt.
- Describing a platform trace as attention, effort, or understanding.
- Treating a forecast as an explanation.
- Recommending intervention before defining possible benefits, harms, and review.

Transfer to another dataset

For any domain, separate the real-world concept from its recorded proxy. A retail click is not satisfaction, a medical claim is not health, and an educational click is not learning. State the proxy, its grain, and its claim boundary.

Chapter summary

Learning analytics uses educational records to answer descriptive and predictive questions, but the records are incomplete proxies. OULAD supports a careful study of registrations, assessments, outcomes, resources, and platform activity. It does not support causal or diagnostic claims by itself. Every result needs an operational definition, an observation grain, and a responsible interpretation boundary.

The OULAD Source Tables and What Each Row Means

Learning objectives

After completing this chapter, you will be able to:

- identify the seven CSV tables used by the project;
- define module, presentation, student attempt, assessment, resource, and interaction in the OULAD context;
- state what one row means (its observation grain) and which columns should identify it (its candidate key) for each source table;
- trace important relationships among the tables;
- interpret relative-day fields and the question-mark missing marker; and
- identify limitations that must remain visible during analysis.

3.1 Read one CSV row before reading seven tables

A CSV file is a text file that represents a table. Consider this simplified excerpt from `courses.csv`:

```
1 code_module,code_presentation,module_presentation_length
2 AAA,2013J,268
```

Listing 3.1: A header and one illustrative CSV data row

Header The first line names the columns. It describes what each position means.

Row The second line is one data record. Here it describes one delivery of module AAA.

Column Values in the same comma-separated position belong to the same variable. The third column records presentation length.

Cell or field One value at the intersection of a row and column, such as 268.

Identifier or ID A value used to name or match a recorded entity. AAA identifies a module, but it is not a person's identity.

The row means that module AAA was delivered in presentation 2013J and that the recorded presentation length is 268 days. Reading a table starts with that sentence, not with calculation.

Validation checkpoint

Point to each part of the sample and answer: Which line is the header? What does the one data row represent? Which value records presentation? Which value is a measurement rather than an identifier?

3.2 Where the data came from: provenance

Provenance means the recorded history of where data came from and how it reached the project. Provenance matters because two files with the same name can contain different bytes or versions.

OULAD was created from anonymized Open University records and released through the UCI Machine Learning Repository [Kuzilek et al., 2015, 2017]. The project downloads the official archive rather than committing the source data. An **archive** is one packaged file containing other files, commonly a ZIP file. **Anonymized** means direct identities have been removed or replaced; it does not mean the records have no privacy risk. The Creative Commons Attribution 4.0 license states the conditions under which the dataset can be reused.

The implemented downloader checks:

- the archive's SHA-256 checksum;
- the presence of all seven CSV files and OULAD.names; and
- the exact expected archive before extraction, which means unpacking the files it contains.

In plain language

A checksum is a fingerprint of a file. If the downloaded bytes change, the fingerprint changes. Matching the expected checksum does not prove that every value is correct, but it does show that the project received the exact archive it was designed to process.

3.3 OULAD vocabulary

Module An anonymized course code, represented as AAA through GGG.

Presentation One delivery of a module in a period such as 2013J or 2014B. The code combines a year with a start term.

Student-module-presentation attempt One student identifier associated with one module and one presentation. A student can appear in more than one attempt.

Assessment A TMA, CMA, or Exam defined for a module-presentation.

VLE resource An online site or activity made available in a module-presentation.

VLE interaction A recorded number of clicks by a student on a site on a course-relative day.

3.4 The seven source files

Table 3.1: Verified OULAD source files used by the project

Source file	Rows	Observation grain	Important identifiers and purpose
<code>courses.csv</code>	22	One module-presentation	Module, presentation, presentation length
<code>assessments.csv</code>	206	One assessment	Assessment ID plus owning module-presentation
<code>vle.csv</code>	6,364	One site in a module-presentation	Site ID, activity type, availability weeks
<code>studentInfo.csv</code>	32,593	One student-module-presentation attempt	Attempt context, demographics, final result
<code>studentRegistration.csv</code>	32,593	One student-module-presentation registration	Registration and unregistration relative days
<code>studentAssessment.csv</code>	173,912	One recorded student-assessment submission	Assessment, student, submitted day, banked flag, score
<code>studentVle.csv</code>	10,655,280	One student-site-relative-day interaction row	Module-presentation, student, site, day, click count

Verified result: The seven CSV files contain 10,900,970 rows in total. These counts come from `reports/tables/source_file_profile.csv`.

3.5 A relational map

The seven files are **related**: a value in one table can be matched with a corresponding value in another. For example, an assessment record names the module and presentation that own it. The arrows in [figure 3.1](#) mean “can be connected through matching identifiers.” They do not mean that every possible connection is automatically correct.

3.6 Grain before keys

Observation grain answers: “What does one row represent?” A **candidate key** answers: “Which columns should uniquely identify that row?” A candidate key is a proposed identifier until the data is tested for duplicates. The grain comes first. Without it, a key is merely a list of columns.

Return to the sample row `AAA,2013J,268`. Its grain is one module presentation. Module alone cannot identify the row because module AAA can be delivered more than once. Module plus presentation is the candidate key.

Validation checkpoint

For every table, say the following sentence aloud:

“One row represents _____, and it should be identified by

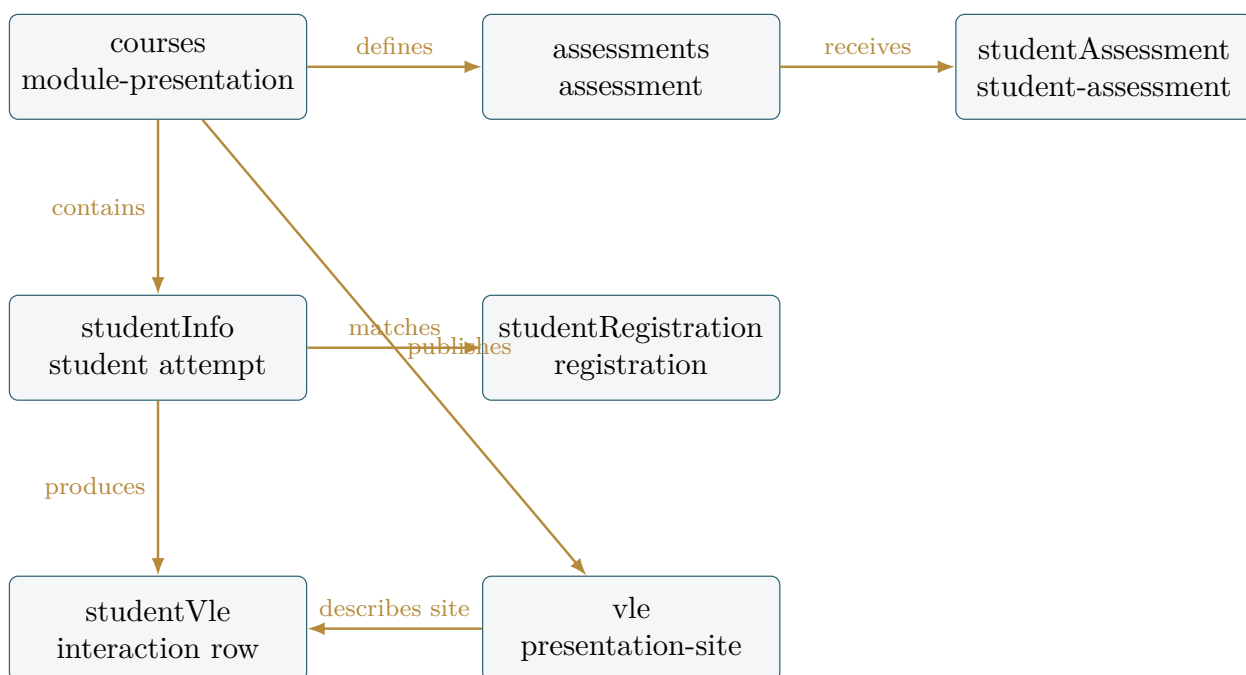


Figure 3.1: Conceptual relationships among the seven source tables. Arrows show how identifiers can connect tables. The original CSV files do not enforce these matches as database rules.

Table 3.2: Source-grain reasoning

Source	Candidate identifying columns	Important caution
courses	module + presentation	Module alone is not unique
studentInfo	module + presentation + student	Student alone is not an attempt key
studentRegistration	module + presentation + student	Must be checked against attempts
assessments	assessment ID	Module context is carried on the assessment
studentAssessment	assessment ID + student	An absent row means no recorded submission
vle	site + module + presentation	Site context belongs to a presentation
studentVle	student + site + module + presentation + day, approximately	Duplicate source-grain rows can occur; the core table deliberately has no declared primary key

_____.”

If you cannot fill both blanks precisely, do not join the table to another one or aggregate it into summaries yet.

3.7 Time is relative

OULAD uses days relative to the start of a presentation rather than calendar dates. Important fields include:

- `date_registration`: registration day relative to course start;
- `date_unregistration`: recorded unregistration day;
- `assessments.date`: expected assessment due day;
- `date_submitted`: recorded submission day; and
- `studentVle.date`: interaction day.

Negative days are meaningful. The saved quality profile finds 688,988 VLE activity rows before day zero. Those rows are not automatically errors: they can represent legitimate pre-course access.

3.8 Missing values and absent rows

Two different forms of missingness matter:

1. A source cell can contain the marker `?`. The staging layer converts it to SQL `NULL`. `NULL` is PostgreSQL’s marker for a missing or unknown value; it is not the number zero or an empty string.
2. A whole expected row can be absent. For example, a student-assessment pair with no matching submission row represents a missing recorded submission in a later analysis table.

These cases must not be handled identically. A missing deprivation-band value is an unknown attribute. A missing submission row is an event defined through the relationship between attempts, assessments, and recorded submissions.

Verified result: The saved audit contains 1,111 missing deprivation-band values and 11 assessments without due dates. The missing due dates are commonly associated with exams, so those assessments are not treated as an eligible “next assessment” when the later project defines a prediction outcome.

3.9 Representational limitations

- Module codes are anonymized, which limits subject-specific interpretation.
- OULAD represents selected Open University offerings, not all education.
- VLE records omit offline study and unrecorded activity.

- Final outcomes and unregistration dates do not contain every reason for an outcome.
- Demographic fields are sensitive contextual variables, not explanations of performance.

3.10 Knowledge check

1. Why is `id_student` not a student-attempt key?
2. What does one row in `studentVle.csv` represent?
3. How is an absent submission row different from a NULL score?
4. Why should negative activity days not be deleted automatically?
5. Which table connects a submission to its module-presentation?

Exercise 3.1: Build the grain and key worksheet

Without looking back at [table 3.2](#), create a seven-row table with these columns:

1. source file;
2. one-row meaning;
3. candidate key;
4. parent table;
5. possible null or absent-row issue; and
6. one analytical question the table can support.

Then compare your table with the project source and correct it. See the guided solution in [appendix A.3.1](#).

Exercise 3.2: Trace one learner attempt

Choose the conceptual path from a student attempt to:

1. its registration;
2. its assessments;
3. its recorded submissions; and
4. its VLE interactions.

List the columns required at each transition and state where row multiplication could occur, meaning one starting row could turn into several matched rows. Do not write the join yet. See the guided solution in [appendix A.3.2](#).

Common mistakes and troubleshooting

- Calling 32,593 attempt rows “32,593 students.” The verified unique student count is 28,785.
- Treating ? as literal text after staging.
- Assuming every assessment has a due day.

- Joining VLE resources by site ID without module-presentation context.
- Treating absence of a submission row as a database import failure.

Transfer to another dataset

The seven-table worksheet generalizes directly to online-store events, health claims, financial transactions, and app-usage logs. Before analysis, map every source's row meaning, identifying columns, related parent record, time representation, and way that values can be missing.

Chapter summary

OULAD contains seven linked sources at different grains. Module, presentation, student, assessment, site, and relative day combine differently across tables. Student ID alone is not an attempt key. Missing cells and absent relationship rows carry different meanings. Reliable analysis begins by defining one row, its identifying columns, and its limitations.

Relational Database Foundations

Learning objectives

After completing this chapter, you will be able to:

- define database, schema, table, row, column, and datatype;
- distinguish primary, foreign, and composite keys;
- reason about one-to-one, one-to-many, and many-to-many relationships;
- explain normalization, referential integrity, views, and indexes;
- predict how a join can change row counts; and
- connect OULAD source grains to the project’s raw, staging, and core layers.

4.1 The relational model

A **relational database** organizes data into relations commonly presented as tables. The relational model provides a disciplined way to identify rows, connect tables, and express transformations [Codd, 1970]. PostgreSQL is the database system used by this project [PostgreSQL Global Development Group, 2026].

Database A named collection of data and database objects. Here it is `learning_analytics`.

Schema A namespace inside a database. The project uses `raw`, `staging`, `core`, `quality`, `analytics`, and `features`.

Table A stored relation made of rows and columns.

Row One observation at the table’s declared grain.

Column One named attribute of every row.

Datatype The allowed representation and operations for a column, such as `text`, `integer`, `numeric`, or `boolean`.

Table 4.1: Database layers and their contracts

Schema	Responsibility	Example
raw	Preserve source columns for loading and reconciliation	Dates and IDs often land as text
staging	Normalize names, types, and missing markers	<code>NULLIF(date, '?')::integer</code>
core	Enforce stable grains, keys, values, and relationships	Composite primary key for attempts
quality	Persist executable checks and severity	Orphan, duplicate, range, and category checks
analytics	Define reusable analytical metrics and marts	Weekly engagement and assessment progress
features	Construct point-in-time model snapshots	One attempt-week with a future eligible assessment

4.2 Why the project has layers

In plain language

The layers separate evidence from interpretation. Raw tables answer “what was in the file?” Staging answers “how should the values be represented?” Core answers “what relationships and rules are dependable?” Later layers answer analytical questions.

4.3 Keys

A **primary key** uniquely identifies a stored row and cannot be null. A **foreign key** requires a value to match a key in another table. A **composite key** uses more than one column.

The implemented attempt table declares:

```

1 CREATE TABLE core.student_attempts (
2     code_module text NOT NULL,
3     code_presentation text NOT NULL,
4     id_student integer NOT NULL,
5     ...
6     PRIMARY KEY (
7         code_module,
8         code_presentation,
9         id_student
10    ),
11    FOREIGN KEY (code_module, code_presentation)
12        REFERENCES core.module_presentations
13        (code_module, code_presentation)
14 );

```

Listing 4.1: Selected key and relationship clauses from `sql/03_core.sql`.

4.3.1 Why the key is composite

A student can appear in multiple modules or presentations. Therefore, `id_student` does not identify an attempt. The three-column key encodes the actual row meaning.

Validation checkpoint

To test a proposed key, group by its columns and search for groups with more than one row:

```
1 SELECT
2     code_module,
3     code_presentation,
4     id_student,
5     count(*) AS rows_at_key
6 FROM staging.student_info
7 GROUP BY 1, 2, 3
8 HAVING count(*) > 1;
```

The saved project run found zero duplicate attempt keys.

4.4 Relationships and cardinality

Cardinality describes how rows in one table relate to rows in another.

One-to-one Each row matches at most one row on the other side. The project expects one registration row for one student attempt.

One-to-many One parent can match many children. A module-presentation defines multiple assessments.

Many-to-many Multiple rows on both sides can match. This relationship generally requires an associative table or careful analytical pre-aggregation.

An attempt can have many assessments and many VLE interaction rows. If those two child tables are joined to the attempt simultaneously without pre-aggregation, each assessment can pair with each interaction. The resulting row multiplication is mathematically valid as a join result but wrong for most analytical questions.

4.4.1 The five join questions

Before every join, answer:

1. What does one row in the left input represent?
2. What does one row in the right input represent?
3. Which columns form the join key?
4. What should one output row represent?
5. How should the output row count change?

SQL joins are taught in the forthcoming chapter, but cardinality reasoning must come first.

4.5 Referential integrity

Referential integrity means that a child reference points to a valid parent. For example, every core assessment's module-presentation must exist in `core.module_presentations`.

Not every source relationship can be declared conveniently as a foreign key. The project supplements core constraints with explicit anti-join checks:

```
1 SELECT count(*) AS orphan_assessments
2 FROM staging.assessments AS assessment
3 LEFT JOIN staging.courses AS course
4     USING (code_module, code_presentation)
5 WHERE course.code_module IS NULL;
```

Listing 4.2: Conceptual form of the implemented orphan-assessment check.

The expected result is zero. A nonzero result means the source contains an assessment whose parent module-presentation is absent.

4.6 Nulls and constraints

SQL NULL means the value is unknown or absent under the database representation. It is not zero and not an empty string. Conditions such as `score = NULL` do not work; use `score IS NULL`.

The core layer adds **constraints**:

- NOT NULL for required values;
- CHECK (`score BETWEEN 0 AND 100`);
- accepted assessment types;
- accepted final-result categories; and
- positive presentation lengths and studied credits.

Constraints prevent invalid future writes. Quality queries also document and profile the source before or beyond what constraints can express.

4.7 Normalization and denormalization

Normalization separates facts so that each fact is stored at an appropriate grain. Module-presentation details, assessments, attempts, resources, submissions, and interactions therefore live in different core tables.

Denormalization deliberately combines or aggregates facts for repeated analytical use. The project's materialized weekly engagement mart is denormalized to one active attempt-week. It is not a replacement for the core tables; it is a derived analytical product.

4.8 Views, materialized views, and indexes

View A saved query evaluated when read. The staging layer uses views so source-faithful raw tables remain available.

Materialized view A saved query whose result rows are stored and later refreshed or rebuilt. The project uses these for expensive analytical marts.

Index An auxiliary structure that can speed selected access patterns while using storage and adding write cost.

These concepts are fully developed in forthcoming SQL chapters. At this stage, remember that an index does not change the meaning or grain of a table.

4.9 Knowledge check

1. Why does the project preserve raw text before casting values?
2. What rule does the attempt composite key enforce?
3. Give one one-to-many OULAD relationship.
4. Explain how joining two child tables can multiply rows.
5. What is the difference between a view and a materialized view?
6. Why are both constraints and quality queries useful?

Exercise 4.1: Diagnose a dangerous join plan

A learner proposes joining attempts, assessments, and VLE interactions in one query using only module and presentation. Before writing SQL:

1. state each input grain;
2. describe the expected multiplication;
3. identify the missing student key where applicable;
4. propose which child data should be pre-aggregated; and
5. define the desired output grain.

See the guided solution in [appendix A.4.1](#).

Exercise 4.2: Classify database rules

For each rule, choose primary key, foreign key, check constraint, null rule, or quality query, and justify the choice:

1. an assessment weight must be between 0 and 100;
2. an interaction site should exist in the same presentation;
3. an attempt is unique by module, presentation, and student;

4. missing deprivation band should remain visible; and
5. the official archive should contain 22 course rows.

See the guided solution in [appendix A.4.2](#).

Common mistakes and troubleshooting

- Declaring a key before defining row grain.
- Believing a successful join preserves the intended row count.
- Replacing unknown values with zero.
- Treating indexes as data-cleaning tools.
- Removing the raw layer after typed tables are built.

Transfer to another dataset

Layered relational design applies beyond OULAD. Preserve a source-faithful landing layer, make type and missingness decisions visible, enforce dependable core contracts, and build denormalized analytical products only after the grain is stable.

Chapter summary

The project uses six PostgreSQL schemas to separate source evidence, typed representations, constrained core data, validation, analytics, and features. Keys express row grain; foreign keys and quality checks express relationships. Nulls, constraints, cardinality, and row multiplication must be reasoned about before analytical SQL is trusted.

Part II

Build the Validated Database

PostgreSQL and Project Setup

Learning objectives

After completing this chapter, you will be able to:

- distinguish the Docker and local PostgreSQL setup paths;
- create an isolated Python 3.11 or newer environment;
- explain the project database URL and its credential fields;
- start or connect to PostgreSQL 16 without exposing credentials;
- verify the database, schemas, Python package, and client tools; and
- diagnose common role, port, environment, and path failures.

5.1 What the environment must provide

The implemented project expects:

- Python 3.11 or newer;
- PostgreSQL 16;
- the `psql` command-line client;
- project Python dependencies; and
- either Docker Compose or a trusted local PostgreSQL installation.

In plain language

Python runs the source audit and later analysis. PostgreSQL owns the relational data and SQL pipeline. The database URL tells Python and shell scripts which PostgreSQL server, database, and role to use.

5.2 Start from the repository root

Every command in this chapter begins at the directory containing `pyproject.toml`, `Makefile`, `sql/`, and `src/`. Verify:

```
1 pwd
2 ls
3 test -f pyproject.toml && echo "repository root found"
```

The final command should print `repository root found`. If it does not, stop and navigate to the Learning Analytics repository before continuing.

5.3 Create the Python environment

The project declares Python 3.11 or newer in `pyproject.toml`. Create a local virtual environment:

```
1 python3 --version
2 python3 -m venv .venv
3 source .venv/bin/activate
4 python -m pip install --upgrade pip
5 python -m pip install -e ".[dev]"
```

On Windows PowerShell, activation is typically:

```
1 .\venv\Scripts\Activate.ps1
```

5.3.1 What the commands do

`python3 -m venv .venv` Creates an isolated environment inside the repository. The environment directory is ignored by Git.

`source .venv/bin/activate` Makes the environment's Python and installed commands active in the current shell.

`pip install -e ".[dev]"` Installs the project in editable mode plus testing and linting dependencies.

Validation checkpoint

Run:

```
1 python --version
2 python -c "import learning_analytics; print('package import ok')"
3 pytest --collect-only -q
```

The Python version must be at least 3.11, the import should succeed, and pytest should discover the project tests.

5.4 Understand the database URL

The project's Docker-oriented default is:

```
1 postgresql://learning_analytics:learning_analytics@localhost:5432/learning_analytics
```

Its structure is:

```
postgresql://ROLE:PASSWORD@HOST:PORT/DATABASE
```

The repeated development credential is suitable only for the local course container. Do not reuse it for an internet-accessible or production database. Do not commit real secrets.

5.5 Path A: Docker Compose

This is the most self-contained project-supported path. It uses `docker-compose.yml` to start PostgreSQL 16 Alpine and create:

- database: `learning_analytics`;
- role: `learning_analytics`;
- password: `learning_analytics`; and
- host port: 5432.

```
1 docker --version
2 docker compose version
3 cp .env.example .env
4 make db-up
5 docker compose ps
```

Wait until the service reports healthy. Then verify the exact configured connection:

```
1 psql \
2   "postgresql://learning_analytics:learning_analytics@localhost:5432/learning_analytics"
3   \
4   -c "select current_database(), current_user;"
```

Expected values are database `learning_analytics` and user `learning_analytics`.

5.6 Path B: Trusted local PostgreSQL

If PostgreSQL 16 is already installed locally and your operating-system user is a trusted PostgreSQL role, the project documents a password-free local socket path:

```
1 createdb learning_analytics
2 export DATABASE_URL=postgresql:///learning_analytics
3 psql "$DATABASE_URL" \
4   -c "select current_database(), current_user, version();"
```

The local role will usually be your operating-system username, not `learning_analytics`. That is acceptable for local study when the database is not exposed externally.

Edition status

Documented implementation ambiguity. The Makefile target `make db-init` runs `createdb learning_analytics`, but it does not create the password role used by the Docker-oriented default URL. On a local installation, export `DATABASE_URL=postgresql:///learning_analytics` as shown above. This course documents the distinction without changing the implementation.

5.7 Environment files versus exported variables

The repository includes `.env.example`. Copying it provides a local credential reference:

```
1 cp .env.example .env
```

The Makefile does not explicitly load `.env`. Do not assume the copy alone changes the current shell. For the local path, use `export DATABASE_URL=...`. For one command only, prefix the variable:

```
1 DATABASE_URL=postgresql:///learning_analytics make validate
```

5.8 Create the project schemas

The import command later runs `sql/00_database.sql`. To inspect the schema contract without changing it:

```
1 CREATE SCHEMA IF NOT EXISTS raw;
2 CREATE SCHEMA IF NOT EXISTS staging;
3 CREATE SCHEMA IF NOT EXISTS core;
4 CREATE SCHEMA IF NOT EXISTS quality;
5 CREATE SCHEMA IF NOT EXISTS analytics;
6 CREATE SCHEMA IF NOT EXISTS features;
```

After import, list them:

```
1 psql "$DATABASE_URL" -c "\dn"
```

5.9 Full setup checkpoint

Do not continue until all items in [table 5.1](#) pass.

5.10 Knowledge check

1. What are the five major parts of a PostgreSQL URL?
2. Why is a virtual environment useful?

Table 5.1: Environment checkpoint

Check	Passing evidence
Repository root	<code>pyproject.toml</code> and <code>sql/</code> are visible
Python version	3.11 or newer
Python package	<code>import learning_analytics</code> succeeds
PostgreSQL client	<code>psql -version</code> succeeds
Server connection	<code>select current_database()</code> returns the expected database
Credential path	Docker URL or local socket URL is explicitly selected
Private values	No personal password is added to tracked files

3. Which setup path creates the `learning_analytics` role?
4. Why might copying `.env.example` not change a Make command?
5. What query verifies both the database and current role?

Exercise 5.1: Produce an environment verification record

Create a text record containing:

1. Python version;
2. PostgreSQL client version;
3. selected setup path;
4. database name and current role;
5. package-import result; and
6. any resolved setup error, without including a password.

The expected output is a short, credential-free environment report. See the guided solution in [appendix A.5.1](#).

Common mistakes and troubleshooting

- **Role does not exist.** You used the Docker URL against a local server. Export the local socket URL or create the intended role yourself.
- **Connection refused.** The server is stopped, the port differs, or Docker is unavailable.
- **Port already in use.** Another PostgreSQL server is already on 5432. Choose one setup path rather than starting both.
- **Package import fails.** Activate the environment and rerun the editable installation.
- **Wrong project root.** Relative paths in Make and shell scripts assume the repository root.

Transfer to another dataset

For another project, record the language version, dependency environment, database engine, connection method, secret-handling rule, and one verification command for each service. A reproducible environment is a documented contract, not merely a list of tools.

Chapter summary

The project supports Docker PostgreSQL with a dedicated development role or a trusted local PostgreSQL socket connection. Python 3.11 or newer and an installed editable package are required. Environment variables must be explicit, credentials must remain local, and every setup stage has a concrete verification command.

Downloading, Auditing, and Importing Raw OULAD Data

Learning objectives

After completing this chapter, you will be able to:

- explain the official archive, checksum, and expected-file contract;
- locate the extracted CSV files without committing raw data;
- explain why most raw columns initially land as text;
- trace the exact table creation and client-side bulk-load order;
- interpret import failure behavior and expected row counts; and
- reconcile the populated raw schema with saved source-audit evidence.

6.1 The raw-data contract

The source data are downloaded on demand. They are not tracked in Git. The expected local paths are:

```
1 data/raw/oulad.zip
2 data/raw/source/assessments.csv
3 data/raw/source/courses.csv
4 data/raw/source/studentAssessment.csv
5 data/raw/source/studentInfo.csv
6 data/raw/source/studentRegistration.csv
7 data/raw/source/studentVle.csv
8 data/raw/source/vle.csv
9 data/raw/source/OULAD.names
```

The downloader in `src/learning_analytics/download.py` verifies an expected SHA-256 checksum and confirms that all eight archive members exist before extraction.

Connection to the OULAD project

The exact archive checksum used by the implementation is stored as a constant in `src/learning_analytics/download.py`. The source auditor then calculates a checksum for each extracted CSV and saves it in `reports/tables/source_file_profile.csv`.

6.2 Download and source audit

With the Python environment active:

```
1 make data
```

This target runs two package commands:

1. `python -m learning_analytics download;`
2. `python -m learning_analytics audit.`

The source audit reads the large CSVs without loading every file into one DataFrame. Its streaming profiler counts rows, records `?` or empty cells, and calculates per-file checksums. Selected smaller files are then read with pandas to calculate unique students, attempt keys, module counts, and observed anomalies.

Validation checkpoint

After `make data`, require:

```
1 test -f data/raw/source/studentVle.csv
2 test -f reports/source-audit.json
3 test -f reports/tables/source_file_profile.csv
```

Then compare the profile with [table 3.1](#). Do not proceed if file names, checksums, or counts differ unexpectedly.

6.3 Why raw tables preserve source shape

The raw schema is a landing zone. In `sql/01_raw_tables.sql`, most columns are declared as `text`. This design has three advantages:

1. values such as `?` can load before null conversion;
2. source contents remain available for reconciliation; and
3. type errors are handled visibly in the staging layer.

The largest table, `raw.student_vle`, uses integer columns already and is declared `UNLOGGED` for load and analytical performance. Unlogged tables reduce write-ahead logging but are not designed for durable source storage across server crashes. The official CSV remains the reproducible source.

```
1 CREATE TABLE raw.assessments (
2     code_module text,
```

```

3      code_presentation text,
4      id_assessment text,
5      assessment_type text,
6      date text,
7      weight text
8 );
9
10 CREATE UNLOGGED TABLE raw.student_vle (
11     code_module text,
12     code_presentation text,
13     id_student integer,
14     id_site integer,
15     date integer,
16     sum_click integer
17 );

```

Listing 6.1: Selected raw landing definitions from `sql/01_raw_tables.sql`.

6.4 Import order

Run:

```
1 make ingest
```

The target delegates to `scripts/load_raw.sh`. The script:

1. creates the six schemas;
2. drops and recreates the seven raw tables;
3. loads each CSV with PostgreSQL `\copy`;
4. creates two supporting raw indexes; and
5. runs `ANALYZE` on the largest raw relations.

6.4.1 Why `\copy` is used

`\copy` is a `psql` client command. It reads a file visible to the client process. Server-side `COPY` reads a path visible to the PostgreSQL server and may require different permissions. Because the script runs from the project machine and constructs absolute project paths, client-side loading is appropriate.

```

1 \copy raw.courses
2 FROM '/absolute/project/path/data/raw/source/courses.csv'
3 WITH (FORMAT csv, HEADER true)

```

Listing 6.2: The project bulk-load pattern.

The course shows a descriptive absolute path above, but the implementation calculates the real path at runtime. Do not hard-code a personal home directory in the project or course source.

6.5 Failure behavior

The shell script begins with:

```
1 set -euo pipefail
```

Every `psql` call uses:

```
1 -v ON_ERROR_STOP=1
```

Together, these settings stop the load when a command fails rather than continuing into an apparently complete database.

Edition status

Suggested enhancement, not implemented: the project does not provide a rejected-row quarantine table. A malformed CSV row stops the bulk load. Edition 0.1 teaches how to inspect the error and source file; a future production-oriented module could add staged quarantine handling.

6.6 Validate the imported row counts

```
1 SELECT 'courses' AS source_table, count(*) AS rows
2 FROM raw.courses
3 UNION ALL
4 SELECT 'assessments', count(*) FROM raw.assessments
5 UNION ALL
6 SELECT 'vle', count(*) FROM raw.vle
7 UNION ALL
8 SELECT 'student_info', count(*) FROM raw.student_info
9 UNION ALL
10 SELECT 'student_registration', count(*)
11 FROM raw.student_registration
12 UNION ALL
13 SELECT 'student_assessment', count(*)
14 FROM raw.student_assessment
15 UNION ALL
16 SELECT 'student_vle', count(*) FROM raw.student_vle
17 ORDER BY source_table;
```

Listing 6.3: Instructional row-count reconciliation query based on the implemented gallery.

The output grain is one row per source table. The sum should be 10,900,970. The expected individual counts are listed in [table 3.1](#).

Validation checkpoint

A correct import requires more than “seven tables exist.” Check:

1. all expected tables exist;
2. each row count matches the source audit;

3. no source file was loaded into the wrong table;
4. the source profile and database counts reconcile; and
5. the next staging step can cast expected values.

6.7 From raw text to staging types

The next pipeline stage creates typed views. For example:

```
1 SELECT
2     trim(code_module) AS code_module,
3     id_assessment::integer AS id_assessment,
4     NULLIF(date, '?')::integer AS due_day,
5     weight::numeric(6, 2) AS weight
6 FROM raw.assessments;
```

Read the expressions from inside out:

1. `NULLIF(date, '?')` converts the missing marker to SQL null;
2. `::integer` casts remaining values to integers;
3. aliases replace source names with analytical names; and
4. `numeric(6,2)` preserves assessment weights with decimal scale.

If an unexpected nonnumeric value appears, the cast fails. That is useful: the source contract has changed or the input is wrong.

6.8 Knowledge check

1. What does the archive checksum verify?
2. Why are most raw columns text?
3. What is the difference between `COPY` and `\copy`?
4. What does `ON_ERROR_STOP` protect against?
5. Why is table existence insufficient import validation?
6. What tradeoff comes with an unlogged table?

Exercise 6.1: Reconcile one imported table

Choose `studentAssessment.csv`. Produce a short reconciliation containing:

1. source path;
2. source-profile row and column counts;
3. raw database row count;

4. raw column names;
5. candidate key;
6. known missing marker rule; and
7. one query that would reveal duplicate candidate keys.

See the guided solution in [appendix A.6.1](#).

Exercise 6.2: Explain a failed cast

Suppose one `assessments.date` value is the text TBD. Explain:

1. why the raw load can succeed;
2. why the staging cast fails;
3. how to locate the source row;
4. why silently converting every invalid value to null is risky; and
5. what evidence should be recorded before deciding on a rule.

See the guided solution in [appendix A.6.2](#).

Common mistakes and troubleshooting

- Running import before download and extraction.
- Loading the same CSV twice without recreating raw tables.
- Confusing `studentInfo` attempts with unique students.
- Treating a successful bulk load as proof of correct column mapping.
- Hard-coding a personal absolute path in SQL.
- Changing a source value to make a cast pass without documenting the evidence.

Transfer to another dataset

The same raw-import pattern works for many datasets: verify the acquired file, preserve a source-faithful landing layer, stop on structural failures, cast in a visible staging layer, and reconcile row counts before analysis.

Chapter summary

The project downloads and checksum-verifies the official OULAD archive, profiles the extracted sources, creates source-faithful raw tables, and bulk-loads seven CSV files with fail-fast client-side commands. Correctness requires reconciliation of files, columns, rows, types, and missing markers, not merely a successful command.

Data Validation as an Executable Contract

Learning objectives

After completing this chapter, you will be able to:

- distinguish validation from cleaning and profiling;
- classify source, key, relationship, range, category, and temporal checks;
- write duplicate, orphan, range, and category validation queries;
- interpret error, warning, and informational severity;
- explain why zero failing rows is meaningful only with a stated rule; and
- use the project's persisted quality results to decide whether analysis may continue.

7.1 Validation is not decoration

Data validation compares observed data with an explicit contract. **Data cleaning** transforms values or records according to a documented rule. **Profiling** describes the data without necessarily declaring it valid or invalid.

In plain language

Profiling says “1,111 values are missing.” Validation says whether that condition violates a rule. Cleaning says what transformation, if any, should follow. The three steps answer different questions.

The project persists every SQL check in `quality.run_results` with:

- run timestamp;
- check name;
- severity;
- number of failing rows;

- status; and
- human-readable detail.

7.2 Severity

Table 7.1: Implemented quality severity contract

Severity	Meaning in this project	Example
Error	A violated condition blocks a trusted pipeline run	Duplicate attempt keys or invalid scores
Warning	An unexpected condition deserves review but may be explainable	Unexpected module or presentation code
Informational	A known condition is profiled and kept visible	Missing deprivation band or pre-course activity

An informational profile is not a silent pass. It records a real feature of the data whose analytical consequences must be considered.

7.3 Source-count checks

The project records fixed row-count expectations for especially important sources:

```

1 SELECT
2     abs((SELECT count(*) FROM raw.courses) - 22)
3     AS failing_rows,
4     CASE
5         WHEN (SELECT count(*) FROM raw.courses) = 22
6         THEN 'pass'
7         ELSE 'fail'
8     END AS status;
```

Listing 7.1: Simplified form of an implemented source-count check.

The analytical question is: “Did the imported source match the verified archive?” The input grain is one course row per module-presentation. The output grain is one check result. A nonzero difference is an error because the database no longer reconciles with the expected archive.

7.4 Natural-key checks

To test the attempt key:

```

1 SELECT count(*) AS duplicate_keys
2 FROM (
3     SELECT
4         code_module,
```

```
5      code_presentation,  
6      id_student  
7  FROM staging.student_info  
8  GROUP BY 1, 2, 3  
9  HAVING count(*) > 1  
10 ) AS duplicated;
```

The inner query returns one row per duplicated key, not one row per duplicated source record. The outer count therefore reports duplicated key groups.

Validation checkpoint

Always ask what `failing_rows` counts. It might count:

- bad source rows;
- duplicated key groups;
- orphan child rows; or
- the absolute difference from an expected count.

The detail column should make the unit explicit.

7.5 Relationship checks with anti-joins

An **orphan** is a child row whose required parent is missing. The project uses a left anti-join pattern:

```
1 SELECT count(*) AS orphan_submissions  
2 FROM staging.assessment_submissions AS submission  
3 LEFT JOIN staging.assessments AS assessment  
4   USING (id_assessment)  
5 WHERE assessment.id_assessment IS NULL;
```

7.5.1 Reasoning through the query

1. The left input is one recorded student-assessment submission.
2. The right input is one assessment.
3. The join key is `id_assessment`.
4. A matching assessment supplies a nonnull right-side ID.
5. The `WHERE` clause retains only unmatched submissions.
6. The final output is one count.

The expected count is zero. A nonzero result means a submission references an assessment absent from the project source model.

7.6 Range and category checks

```

1 SELECT count(*) AS invalid_scores
2 FROM staging.assessment_submissions
3 WHERE score NOT BETWEEN 0 AND 100;
4
5 SELECT count(*) AS invalid_results
6 FROM staging.student_info
7 WHERE final_result NOT IN (
8     'Distinction', 'Pass', 'Fail', 'Withdrawn'
9 );

```

Listing 7.2: Implemented validation patterns for scores and outcomes.

`BETWEEN 0 AND 100` includes both boundaries. `NOT IN` is useful for a documented finite set, but null values need separate reasoning because SQL’s three-valued logic does not classify null as “not in” the set.

7.7 Date and temporal checks

The raw dates are course-relative integers. The project checks that unregistration does not precede registration:

```

1 SELECT count(*) AS invalid_registration_order
2 FROM staging.student_registration
3 WHERE unregistration_day < registration_day;

```

The later snapshot test checks that the latest activity used in a feature is no later than the snapshot:

```

1 SELECT count(*) AS future_information
2 FROM features.model_snapshots
3 WHERE latest_feature_day > snapshot_day;

```

The second query previews the forthcoming leakage-prevention chapter. It is shown here because it is part of the real 24-check quality result, not because time-aware modeling is fully taught in Edition 0.1.

7.8 Known profiles are evidence

The saved project run reports:

Verified result: The saved SQL quality table contains 24 results: 18 error-level checks, three warnings, and three informational profiles. No error-level check failed in that run.

7.9 Run the quality gate

After raw import and the ordered SQL pipeline:

```

1 make sql
2 make validate

```

Table 7.2: Selected persisted quality profiles

Check	Failing/profiled rows	Interpretation
Error-level checks	0 across 18 checks	The recorded error contracts passed
Activity before course start	688,988	Legitimate negative relative days; excluded from model weeks
Missing deprivation band	1,111	Missing context remains visible
Missing assessment due dates	11	Often exams; excluded from next-assessment targets
Unexpected module codes	0	Codes stayed within AAA–GGG
Unexpected presentation codes	0	Codes matched documented year/term format

`scripts/run_quality.sh` reruns source and relational checks. If the snapshot table exists, it also runs the four point-in-time checks. It then counts failed error-level results and exits with a failure status when the count is nonzero.

Validation checkpoint

The gate passes only when:

1. the shell command exits successfully;
2. every error-level row has status **pass**;
3. warnings and informational profiles have been read;
4. the run timestamp is current for the database under study; and
5. no expected check silently disappeared.

7.10 How to investigate a failed check

Use a consistent procedure:

1. Identify the check name and what its failing count represents.
2. Open the implementing SQL and isolate the detail query.
3. Replace the outer count with selected identifying columns.
4. Inspect a small bounded sample without editing source data.
5. Compare with official documentation and source audit.
6. Decide whether the issue is import, source, representation, or rule.
7. Document the decision before any cleaning change.

Edition status

This course does not change project quality rules. If a rule appears incomplete or incorrect, record the evidence as a suggested enhancement. Do not silently weaken the check to make the pipeline pass.

7.11 Knowledge check

1. Distinguish profiling, validation, and cleaning.
2. What does an anti-join find?
3. Why can `NOT IN` require a separate null check?
4. What does zero mean in the duplicate-attempt query?
5. Why must warnings be read even when the gate exits successfully?
6. What is the first step after a failed quality check?

Exercise 7.1: Write a validation specification

For each rule below, write its severity, input grain, failing-row meaning, expected result, and investigation query:

1. assessment weights are between 0 and 100;
2. every VLE interaction site exists in the same module-presentation;
3. no attempt key is duplicated;
4. deprivation band may be missing but must be profiled; and
5. unregistration cannot precede registration.

See the guided solution in [appendix A.7.1](#).

Exercise 7.2: Debug an orphan count

Suppose the orphan-submission check returns 12. Explain how to turn the count query into a diagnostic query, which identifiers to display, which source files to inspect, and why deleting the 12 rows is not an acceptable first response. See the guided solution in [appendix A.7.2](#).

Common mistakes and troubleshooting

- Treating every nonzero profile as an error.
- Treating a zero count as meaningful without understanding its unit.
- Changing source data before deciding whether the rule or import is wrong.
- Ignoring null behavior in range or membership conditions.
- Reporting an old persisted quality run as evidence for a new database.

Transfer to another dataset

Build validation around contracts that name a grain, rule, severity, failing unit, and expected response. Persist results with timestamps. This pattern works for event logs, financial ledgers, clinical records, and any other multi-table analytical system.

Chapter summary

Validation turns assumptions into executable, persisted evidence. The project checks source counts, natural keys, relationships, ranges, categories, dates, and future-information constraints. Error, warning, and informational results have different meanings. A successful gate is useful only when the checks, failing units, timestamps, and known profiles are understood.

Part III

Query the Evidence

SQL Foundations with Real OULAD Questions

Learning objectives

After completing this chapter, you will be able to:

- use **SELECT**, **FROM**, aliases, **LIMIT**, and **ORDER BY**;
- filter with comparisons, logical operators, **IN**, **BETWEEN**, **LIKE**, and **IS NULL**;
- construct categories with **CASE**;
- use **DISTINCT**, **COUNT**, **SUM**, **AVG**, **MIN**, and **MAX**;
- aggregate with **GROUP BY** and filter groups with **HAVING**; and
- state and validate the input and output grain of every query.

8.1 The shape of a query

A SQL query is a request for a relation. Introductory queries often follow:

```
1 SELECT column_or_expression
2 FROM schema.table
3 WHERE row_condition
4 GROUP BY grouping_columns
5 HAVING group_condition
6 ORDER BY sorting_expression
7 LIMIT number_of_rows;
```

SQL is written in this conventional order, although the database reasons about the clauses in a different logical sequence. Begin by identifying **FROM**: it tells you the input table and therefore the input grain.

8.2 Inspect a table safely

8.2.1 Question

What columns and example values appear in the raw course table?

```

1 SELECT
2     code_module,
3     code_presentation,
4     module_presentation_length
5 FROM raw.courses
6 ORDER BY code_module, code_presentation
7 LIMIT 5;

```

Listing 8.1: Instructional inspection query.

8.2.2 Reasoning

- **Input grain:** one module-presentation.
- **Output grain:** one module-presentation, capped at five rows.
- **SELECT:** names the returned columns.
- **FROM:** names the schema-qualified table.
- **ORDER BY:** makes the limited sample deterministic.
- **LIMIT:** protects against accidentally printing a large table.

Validation checkpoint

LIMIT 5 does not prove that the table contains five rows or that the sample represents the full distribution. Verify total row count separately.

8.3 Count rows and distinct entities

8.3.1 Question

How many course rows, anonymized modules, attempts, and unique students exist?

```

1 SELECT
2     count(*) AS module_presentation_rows,
3     count(DISTINCT code_module) AS modules
4 FROM raw.courses;
5
6 SELECT
7     count(*) AS student_module_attempts,
8     count(DISTINCT id_student) AS unique_students,
9     count(*) - count(DISTINCT id_student)
10    AS additional_attempt_records
11 FROM core.student_attempts;

```

Verified result: The expected results are 22 module-presentations, seven modules, 32,593 attempts, 28,785 unique students, and 3,808 additional attempt records.

8.3.2 Why the two counts differ

`count(*)` counts rows. `count(DISTINCT id_student)` counts different nonnull identifiers. Their difference proves that some identifiers appear in more than one attempt. It does not tell you whether the repeat is another module, another presentation, or both.

8.4 Filter rows

8.4.1 Comparisons and logical operators

```
1 SELECT
2     code_module,
3     code_presentation,
4     id_student,
5     registration_day
6 FROM core.student_attempts
7 WHERE registration_day < 0
8     AND code_module IN ('AAA', 'BBB')
9 ORDER BY registration_day
10 LIMIT 25;
```

The query asks for attempts registered before day zero in either module AAA or BBB. `AND` requires both conditions. `IN` is a readable form of multiple equality alternatives.

8.4.2 BETWEEN and LIKE

```
1 SELECT
2     code_module,
3     code_presentation,
4     id_student,
5     activity_day,
6     click_count
7 FROM core.vle_interactions
8 WHERE activity_day BETWEEN 0 AND 6
9     AND code_presentation LIKE '2014%'
10 ORDER BY code_module, code_presentation, id_student
11 LIMIT 100;
```

`BETWEEN 0 AND 6` includes both endpoints. The pattern `'2014%'` means text beginning with 2014. The query's input and output grain remain interaction rows; filtering changes which rows appear, not what one row represents.

8.5 Work with null values

8.5.1 Question

How many attempt records have no deprivation-band value?

```

1 SELECT count(*) AS missing_imd_band
2 FROM staging.student_info
3 WHERE imd_band IS NULL;

```

Verified result: The saved quality profile reports 1,111 rows.

Do not write `imd_band = NULL`. SQL null represents unknown or absent information and must be tested with `IS NULL` or `IS NOT NULL`.

8.6 Create an interpretable category with CASE

8.6.1 Question

Can registration days be summarized into a few timing bands?

```

1 SELECT
2     CASE
3         WHEN registration_day IS NULL THEN 'unknown'
4         WHEN registration_day < 0 THEN 'before course start'
5         WHEN registration_day BETWEEN 0 AND 13 THEN 'first two weeks'
6         ELSE 'later'
7     END AS registration_timing,
8     count(*) AS attempts
9 FROM core.student_attempts
10 GROUP BY registration_timing
11 ORDER BY attempts DESC;

```

Listing 8.2: Instructional categorical transformation.

CASE is evaluated from top to bottom. The explicit null branch matters: a null comparison is not true, so an omitted branch would place unknown values in ELSE.

Validation checkpoint

Sum the grouped `attempts`. It should equal the total attempt count, 32,593. If it does not, investigate whether rows were filtered or a group was lost.

8.7 Aggregate outcomes

8.7.1 Question

How many attempts ended in each recorded final result?

```

1 SELECT
2     final_result,
3     count(*) AS attempts,
4     round(
5         100.0 * count(*) / sum(count(*) OVER (),
6         1
7     ) AS percent_of_attempts

```

```

8 FROM core.student_attempts
9 GROUP BY final_result
10 ORDER BY attempts DESC;

```

Table 8.1: Verified recorded outcome counts

Final result	Attempts	Approximate percent
Pass	12,361	37.9
Withdrawn	10,156	31.2
Fail	7,052	21.6
Distinction	3,024	9.3

The input grain is one attempt. The output grain is one final-result category. The grouped counts must sum to 32,593. Percentages may sum to 99.9 or 100.1 after rounding.

8.8 Use numerical aggregates

8.8.1 Question

What is the range and average of recorded nonnull assessment scores?

```

1 SELECT
2     count(score) AS recorded_scores,
3     min(score) AS minimum_score,
4     max(score) AS maximum_score,
5     round(avg(score), 2) AS mean_score
6 FROM core.assessment_submissions;

```

- `count(score)` excludes null scores.
- `min` and `max` establish observed boundaries.
- `avg` excludes nulls rather than treating them as zero.
- The output grain is one summary row.

The project enforces recorded scores between 0 and 100. This query should not be used to infer the number of expected submissions because absent submissions have no row in this table.

8.9 Group by module-presentation

8.9.1 Question

Which module-presentations have at least 1,000 attempt records?

```

1 SELECT
2     code_module,

```

```

3     code_presentation,
4     count(*) AS attempts
5 FROM core.student_attempts
6 GROUP BY code_module, code_presentation
7 HAVING count(*) >= 1000
8 ORDER BY attempts DESC;

```

WHERE filters input rows before grouping. HAVING filters groups after aggregation. The output grain is one retained module-presentation, not one attempt.

8.10 Conditional aggregation

8.10.1 Question

How many outcomes of each type occur within a module-presentation?

```

1 SELECT
2     code_module,
3     code_presentation,
4     count(*) AS student_attempts,
5     count(*) FILTER (
6         WHERE final_result = 'Pass'
7     ) AS passes,
8     count(*) FILTER (
9         WHERE final_result = 'Withdrawn'
10    ) AS withdrawals,
11    round(
12        avg((final_result = 'Withdrawn')::integer)::numeric,
13        4
14    ) AS withdrawal_rate
15 FROM core.student_attempts
16 GROUP BY code_module, code_presentation;

```

Listing 8.3: Selected production pattern from `analytics.course_outcome_summary`.

PostgreSQL's `FILTER` clause calculates multiple conditional aggregates without changing the group grain. Casting the Boolean comparison to integer turns false into 0 and true into 1; its average is therefore a proportion.

8.11 Validation habits for introductory SQL

For every query, record:

8.12 Knowledge check

1. Why should `ORDER BY` accompany a reproducible limited sample?
2. What is the difference between `count(*)` and `count(score)`?
3. When is `HAVING` used instead of `WHERE`?

Table 8.2: SQL reasoning template

Question	Required statement
Analytical question	What decision or understanding motivates the query?
Input	Which table or view is read?
Input grain	What does one input row represent?
Filter	Which rows are included or excluded?
Output grain	What does one result row represent?
Null risk	Which expressions exclude or preserve nulls?
Duplication risk	Can one input row appear more than once?
Expected evidence	Which counts, ranges, or categories are known?
Validation	How will row counts or totals be reconciled?
Later use	Which analysis could consume the result?

4. Why does filtering not automatically change table grain?
5. What null error can occur in a `CASE` expression?
6. How can grouped totals validate an aggregation?

Exercise 8.1: Inspect assessment definitions

Write a query returning module, presentation, assessment type, due day, and weight from `core.assessments`. Keep non-exam assessments with a known due day between 0 and 90, order by module-presentation and due day, and limit the display to 50 rows. State input grain, output grain, null rule, and expected use. See the guided solution in [appendix A.8.1](#).

Exercise 8.2: Validate outcome percentages

Extend the outcome count query to return a percentage. Verify that counts sum to 32,593 and explain why rounded percentages might not sum to exactly 100. See the guided solution in [appendix A.8.2](#).

Exercise 8.3: Profile missingness by module

Write a query that returns one row per module with:

- total attempts;
- attempts with missing `imd_band`; and
- missing percentage.

Use conditional aggregation. State why this is descriptive and not evidence that module membership caused missingness. See the guided solution in [appendix A.8.3](#).

Exercise 8.4: Find repeated student identifiers

Write a query returning student IDs that occur in more than one attempt, including their number of attempts. Order from most to least attempts and limit the display to 25. Explain why this output must not be joined back to interaction rows without a module-presentation key. See the guided solution in [appendix A.8.4](#).

Common mistakes and troubleshooting

- Running `SELECT *` on the 10.6 million-row interaction table without a limit.
- Omitting `ORDER BY` and treating an arbitrary sample as stable.
- Using `= NULL`.
- Treating null scores as zero without an explicit analytical rule.
- Selecting nongrouped columns that do not define the output grain.
- Interpreting grouped associations as causes.

Transfer to another dataset

The SQL clauses in this chapter transfer directly to most relational datasets. What changes is the domain meaning: table grain, valid categories, missingness, and validation totals. Carry the reasoning template with the syntax.

Chapter summary

Introductory SQL begins with a question and an input grain. Selection, filtering, categorization, distinct counting, numerical aggregation, grouping, and group filtering produce relations at explicit output grains. Null behavior and grouped reconciliation are part of the query, not afterthoughts. The next SQL chapters will build joins and windows on this foundation.

Cumulative Lab: From Source Files to Defensible Findings

In plain language

This lab brings the first eight chapters together. You will not train a model or claim that engagement causes an outcome. You will build something more fundamental: a reproducible path from documented source files to a small set of carefully checked descriptive findings.

9.1 Learning objectives

By the end of the lab, you should be able to:

- state an analytical question and identify its unit of analysis;
- explain the grain and key of every source table you use;
- establish a PostgreSQL environment without hiding configuration choices;
- audit, import, type, and validate the OULAD source data;
- write introductory SQL that answers bounded descriptive questions; and
- keep an evidence log that separates observed results from interpretation.

9.2 Scenario and deliverables

Imagine that a learning-support team wants an initial orientation to the data. The team asks:

How many student–module attempts appear in each final-result category, how often do students appear in more than one module presentation, and which source quality conditions should constrain later analysis?

The wording matters. This is descriptive, not causal. It asks what the recorded data contain, not why a student passed, failed, or withdrew.

Your lab submission has five small deliverables:

1. a one-page source and grain map;
2. an environment record containing the database path you used;
3. an import reconciliation table;
4. a quality-gate note; and
5. a six-query SQL portfolio with an evidence log.

Connection to the OULAD project

The lab’s primary unit. The primary unit is a *student–module presentation attempt*, represented by one row in `studentInfo.csv`. The same student may contribute more than one attempt. A count of rows is therefore not automatically a count of distinct people.

9.3 Part A: Write the source and grain map

Before opening SQL, complete [table 9.1](#). The first two rows are provided as examples.

Table 9.1: Cumulative lab source and grain map

Source	One row represents	Candidate key	Intended role
<code>studentInfo.csv</code>	One student in one module presentation	<code>student_id + module + presentation</code>	Attempt-level demographics and final result
<code>studentRegistration.csv</code>	One registration record for an attempt	<code>student_id + module + presentation</code>	Registration and withdrawal timing
<code>courses.csv</code>	<i>Complete</i>	<i>Complete</i>	<i>Complete</i>
<code>assessments.csv</code>	<i>Complete</i>	<i>Complete</i>	<i>Complete</i>
<code>studentAssessment.csv</code>	<i>Complete</i>	<i>Complete</i>	<i>Complete</i>
<code>vle.csv</code>	<i>Complete</i>	<i>Complete</i>	<i>Complete</i>
<code>studentVle.csv</code>	<i>Complete</i>	<i>Complete</i>	<i>Complete</i>

Validation checkpoint

For every table, say the phrase “one row represents” aloud. If you cannot finish the sentence precisely, return to [chapter 3](#). A list of column names is not a grain statement.

Lab checkpoint 1: defend the unit of analysis

Write three sentences explaining why `COUNT(*)` from `core.student_info` counts attempts, why `COUNT(DISTINCT student_id)` counts people, and why the two values need not match. Then predict which count will be larger in this dataset.

9.4 Part B: Record the environment

Choose one of the supported paths from [chapter 5](#): Docker Compose or a local PostgreSQL installation. Record:

- the path chosen;
- the PostgreSQL version;
- the database name;
- the exact form of DATABASE_URL, with any password redacted;
- the command used to prove the connection; and
- the date and time of the run.

```
1 python --version
2 psql --version
3 psql "$DATABASE_URL" -X -v ON_ERROR_STOP=1 \
4 -c "select current_database(), current_user, version();"
```

Listing 9.1: A concise environment record

Common mistakes and troubleshooting

Do not publish secrets. Record the *shape* of a connection string, but redact passwords and private host details before publishing screenshots, logs, or coursework.

Lab checkpoint 2: diagnose before proceeding

Suppose `psql` reports that the role `learning_analytics` does not exist. Give one repair for the Docker path and one repair for a trusted local PostgreSQL path. Explain why repeatedly rerunning the import script would not solve this connection error.

9.5 Part C: Audit and import

Follow the repository’s intended sequence:

1. obtain the source archive through the audited download command;
2. verify the expected files and source checksum;
3. initialize the schemas and raw tables;
4. load all seven CSV files;
5. build the staging and core relations; and
6. reconcile source, raw, and core counts.

Do not skip the source audit merely because the filenames look right. A reproducible source is identified by content and structure, not only by a familiar name.

```

1 make setup
2 make data
3 make db-init
4 make load
5 make pipeline
6 make quality

```

Listing 9.2: Repository-level orchestration for a complete first run

The exact targets may be run separately during diagnosis. The shell scripts use stop-on-error behavior, and the SQL entry points use `ON_ERROR_STOP`; a failed statement should interrupt the run rather than quietly producing a partly built database.

9.5.1 Reconciliation table

Complete one row per source:

Table 9.2: Import reconciliation record

Source	CSV rows	Raw rows	Core rows	Difference	Explanation
<code>courses.csv</code>	22				
<code>assessments.csv</code>	206				
<code>vle.csv</code>	6,364				
<code>studentInfo.csv</code>	32,593				
<code>studentRegistration.csv</code>	32,593				
<code>studentAssessment.csv</code>	173,912				
<code>studentVle.csv</code>	10,655,280				

For the current source, the seven CSV files contain 10,900,970 data rows in total. Treat that number as a reconciliation target, not as proof that every row is analytically valid.

Lab checkpoint 3: investigate a discrepancy

Assume `raw.student_assessment` has 173,911 rows after import, while the audited CSV has 173,912. List a three-step investigation that preserves the failed evidence. Do not propose manually inserting a guessed row.

9.6 Part D: Interpret the quality gate

Run the repository quality suite and classify the output:

Errors Conditions that must pass before analysis continues.

Warnings Conditions that require interpretation or a documented choice.

Information Profiles that describe the data without declaring failure.

The checked project results contain 24 quality entries: 18 error-level tests that pass, three warning profiles, and three informational profiles. The warnings identify 688,988 activity records dated before module start, 11 assessments with missing due dates, and 1,111 attempt records with missing IMD-band values.

Lab checkpoint 4: write a quality-gate note

Write a five-sentence quality-gate note. It must state whether the error gate passed, describe all three warning profiles, distinguish a warning from a failure, and name one later analytical decision affected by each warning.

9.7 Part E: Build the six-query portfolio

Run each query, save the SQL, and add one evidence-log entry. The queries are small by design: the challenge is precise interpretation.

9.7.1 Query 1: confirm the attempt population

```
1 SELECT
2     COUNT(*) AS attempt_count,
3     COUNT(DISTINCT student_id) AS student_count
4 FROM core.student_info;
```

Expected values for the audited source are 32,593 attempts and 28,785 distinct students.

9.7.2 Query 2: describe final outcomes

```
1 SELECT
2     final_result,
3     COUNT(*) AS attempts,
4     ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct
5 FROM core.student_info
6 GROUP BY final_result
7 ORDER BY attempts DESC;
```

The expected counts are Pass 12,361, Withdrawn 10,156, Fail 7,052, and Distinction 3,024. State explicitly that these are attempt counts.

9.7.3 Query 3: count repeat appearances

```
1 SELECT
2     COUNT(*) FILTER (WHERE attempt_count = 1) AS one_attempt,
3     COUNT(*) FILTER (WHERE attempt_count > 1) AS multiple_attempts,
4     MAX(attempt_count) AS maximum_attempts
5 FROM (
6     SELECT student_id, COUNT(*) AS attempt_count
7     FROM core.student_info
8     GROUP BY student_id
9 ) AS per_student;
```

9.7.4 Query 4: inspect withdrawal timing

```

1 SELECT
2     COUNT(*) AS registration_records,
3     COUNT(date_unregistration) AS recorded_withdrawal_dates,
4     MIN(date_unregistration) AS earliest_relative_day,
5     MAX(date_unregistration) AS latest_relative_day
6 FROM core.student_registration;

```

The day fields are relative to presentation start. Do not silently translate them into calendar dates.

9.7.5 Query 5: profile assessment due-date missingness

```

1 SELECT
2     assessment_type,
3     COUNT(*) AS assessments,
4     COUNT(*) FILTER (WHERE due_date IS NULL) AS missing_due_dates
5 FROM core.assessments
6 GROUP BY assessment_type
7 ORDER BY assessment_type;

```

9.7.6 Query 6: bound engagement timing

```

1 SELECT
2     COUNT(*) AS activity_rows,
3     COUNT(*) FILTER (WHERE activity_date < 0) AS pre_start_rows,
4     MIN(activity_date) AS earliest_day,
5     MAX(activity_date) AS latest_day
6 FROM core.student_vle;

```

Common mistakes and troubleshooting

A row is not a behavior. An activity record reflects the structure of the VLE extract. It is not, without further definition, a learning session, a moment of attention, or evidence of motivation.

9.8 The evidence log

Use one row per reported finding:

A strong entry for Query 2 might read: “Among 32,593 recorded student–module presentation attempts, 10,156 (31.2%) have a final result of Withdrawn. This describes recorded outcomes at attempt grain; it does not estimate a causal effect or a person-level withdrawal rate.”

Lab checkpoint 5: final synthesis

Write a 150–200 word briefing for the learning-support team. Include two descriptive findings, one quality warning, one grain warning, and one next question. Avoid the words *caused*, *proved*, and *at risk* unless the evidence genuinely supports them.

Table 9.3: Evidence-log template

Query	Observed result	Unit and population	Interpretation	Limitation or check
1				
2				
3				
4				
5				
6				

9.9 Common mistakes

- Reporting an attempt count as a student count.
- Treating an expected row count as sufficient evidence of source integrity.
- Continuing after a failed error-level quality check.
- Deleting pre-start activity because its date looks surprising.
- Calling missing values “bad data” before studying their meaning.
- Describing a descriptive association as an explanation or intervention effect.
- Publishing credentials or machine-specific paths in an evidence log.

9.10 Transfer task

Choose a different relational dataset with at least three tables. Produce only the source-and-grain map and the six fields of an environment record. Then write two introductory aggregation queries. The point is to transfer the workflow, not the OULAD vocabulary.

9.11 Guided solutions

Guided responses for the five checkpoints appear in [appendices A.9.1 to A.9.5](#).

9.12 Chapter summary

The cumulative lab turns a collection of commands into an analytical discipline: define the question, name the grain, record the environment, audit the source, reconcile the import, pass the quality gate, query deliberately, and preserve the boundary between evidence and interpretation.

Part IV

Forthcoming Course Architecture

Forthcoming Course Modules

In plain language

Edition 0.1 ends after introductory SQL and the cumulative lab. The modules in this chapter are a transparent roadmap, not claims of finished instruction. Each entry names the intended learning work and the project evidence that a later edition should use.

SQL Joins and Analytical Grain

Edition status

Forthcoming after Course Edition 0.1. This chapter is part of the approved course architecture, but its lessons, exercises, and solutions have not yet been written. It is not represented as completed work.

Planned learning objectives

Join the OULAD relations without changing the intended unit of analysis; reason about one-to-one, one-to-many, and many-to-many paths; detect row multiplication; and validate pre-aggregation.

Planned practical work

Construct an attempt-level assessment summary and an attempt-level activity summary, then join them only after key tests pass.

Existing project sources

`sql/02_staging.sql`, `sql/03_core.sql`, and `sql/gallery/`.

Intermediate and Advanced SQL

Edition status

Forthcoming after Course Edition 0.1. This chapter is part of the approved course architecture, but its lessons, exercises, and solutions have not yet been written. It is not represented as completed work.

Planned learning objectives

Use common table expressions, window functions, conditional aggregation, date-relative frames, query plans, views, and materialized views while preserving analytical meaning.

Planned practical work

Build presentation-level rankings and cumulative engagement summaries, then compare equivalent query plans.

Existing project sources

`sql/gallery/`, `sql/06_features.sql`, and existing reporting queries.

Python and pandas

Edition status

Forthcoming after Course Edition 0.1. This chapter is part of the approved course architecture, but its lessons, exercises, and solutions have not yet been written. It is not represented as completed work.

Planned learning objectives

Load database extracts safely, inspect dtypes and keys, write vectorized transformations, test reusable functions, and compare pandas operations with their SQL equivalents.

Planned practical work

Reproduce one validated SQL summary in pandas and reconcile every count.

Existing project sources

`src/learning_analytics/config.py`, `src/learning_analytics/analysis.py`, and the test suite.

Exploratory Data Analysis

Edition status

Forthcoming after Course Edition 0.1. This chapter is part of the approved course architecture, but its lessons, exercises, and solutions have not yet been written. It is not represented as completed work.

Planned learning objectives

Separate data checks from exploration; examine distributions, missingness, time, and subgroup composition; and maintain a question-and-evidence log.

Planned practical work

Create an attempt-level EDA notebook or script whose reported totals reconcile to the quality outputs.

Existing project sources

- `reports/tables/source_file_profile.csv`
- `reports/tables/sql_quality_results.csv`
- Existing analysis code

Statistics

Edition status

Forthcoming after Course Edition 0.1. This chapter is part of the approved course architecture, but its lessons, exercises, and solutions have not yet been written. It is not represented as completed work.

Planned learning objectives

Use sampling language, uncertainty intervals, effect sizes, and carefully bounded comparisons; distinguish statistical association from practical or causal importance.

Planned practical work

Estimate and interpret uncertainty for selected descriptive proportions without presenting the source as a random sample of all learners.

Existing project sources

Outcome summaries, the OULAD documentation, and later analysis outputs.

Visualization

Edition status

Forthcoming after Course Edition 0.1. This chapter is part of the approved course architecture, but its lessons, exercises, and solutions have not yet been written. It is not represented as completed work.

Planned learning objectives

Choose visual encodings that match data type and question, expose denominators, handle missing categories, and test charts for accessibility and interpretive honesty.

Planned practical work

Rebuild the outcome distribution and one time-based view with a written chart-validation checklist.

Existing project sources

`reports/figures/` and `src/learning_analytics/analysis.py`.

Engagement Feature Engineering

Edition status

Forthcoming after Course Edition 0.1. This chapter is part of the approved course architecture, but its lessons, exercises, and solutions have not yet been written. It is not represented as completed work.

Planned learning objectives

Define activity windows, aggregate clicks and active days, distinguish site breadth from volume, and test feature grain.

Planned practical work

Build attempt-level engagement features for explicitly bounded observation windows and reconcile them to source activity.

Existing project sources

`sql/06_features.sql` and `src/learning_analytics/features.py`.

Assessment Feature Engineering

Edition status

Forthcoming after Course Edition 0.1. This chapter is part of the approved course architecture, but its lessons, exercises, and solutions have not yet been written. It is not represented as completed work.

Planned learning objectives

Construct submission, score, deadline, and completion features while preserving missingness and avoiding duplicated assessments.

Planned practical work

Produce an attempt-level assessment feature table with key, range, and denominator tests.

Existing project sources

`sql/06_features.sql`, assessment source tables, and feature code.

Time-aware Analytical Design

Edition status

Forthcoming after Course Edition 0.1. This chapter is part of the approved course architecture, but its lessons, exercises, and solutions have not yet been written. It is not represented as completed work.

Planned learning objectives

Define an observation date, prediction horizon, eligibility rule, and outcome window; align every feature to information available at that time.

Planned practical work

Draw and implement a timeline for one early-support analysis.

Existing project sources

Relative-day fields, registration timing, VLE activity, and assessment due dates.

Leakage Prevention

Edition status

Forthcoming after Course Edition 0.1. This chapter is part of the approved course architecture, but its lessons, exercises, and solutions have not yet been written. It is not represented as completed work.

Planned learning objectives

Recognize target, temporal, aggregation, and split leakage; audit feature provenance; and write automated leakage checks.

Planned practical work

Classify candidate features by availability and reject those containing post-cutoff or outcome-derived information.

Existing project sources

`docs/methodology.md`, feature SQL, and modeling code.

Predictive Modeling

Edition status

Forthcoming after Course Edition 0.1. This chapter is part of the approved course architecture, but its lessons, exercises, and solutions have not yet been written. It is not represented as completed work.

Planned learning objectives

Establish a baseline, define a target and eligible cohort, fit interpretable models, and keep preprocessing inside the training workflow.

Planned practical work

Compare a naive baseline with a regularized classifier using a time-aware split.

Existing project sources

`src/learning_analytics/modeling.py` and model configuration.

Model Evaluation

Edition status

Forthcoming after Course Edition 0.1. This chapter is part of the approved course architecture, but its lessons, exercises, and solutions have not yet been written. It is not represented as completed work.

Planned learning objectives

Choose metrics for class imbalance and decision context; evaluate discrimination, calibration, thresholds, and subgroup behavior.

Planned practical work

Create a model-comparison table whose metrics are tied to a stated use case rather than a single winning score.

Existing project sources

Saved model metrics, prediction outputs, and `docs/model-card.md`.

Error Analysis and Interpretation

Edition status

Forthcoming after Course Edition 0.1. This chapter is part of the approved course architecture, but its lessons, exercises, and solutions have not yet been written. It is not represented as completed work.

Planned learning objectives

Inspect false positives and false negatives, compare errors across relevant groups and presentations, and separate model behavior from causal stories.

Planned practical work

Build a reproducible error slice and write an interpretation with competing explanations.

Existing project sources

Prediction outputs, attempt attributes, and model documentation.

Responsible Educational Analytics

Edition status

Forthcoming after Course Edition 0.1. This chapter is part of the approved course architecture, but its lessons, exercises, and solutions have not yet been written. It is not represented as completed work.

Planned learning objectives

Analyze privacy, consent, fairness, intervention burden, human oversight, and the consequences of acting on uncertain predictions.

Planned practical work

Write a pre-deployment review that includes a non-deployment option and an appeals path.

Existing project sources

`docs/model-card.md`, dataset documentation, and published OULAD context.

Reproducibility and Software Quality

Edition status

Forthcoming after Course Edition 0.1. This chapter is part of the approved course architecture, but its lessons, exercises, and solutions have not yet been written. It is not represented as completed work.

Planned learning objectives

Package configuration, tests, data contracts, deterministic builds, logs, and documentation into an auditable analytical product.

Planned practical work

Rebuild the project from a clean environment and produce a provenance manifest for every reported artifact.

Existing project sources

Makefile, `pyproject.toml`, scripts, SQL entry points, and tests.

Final Capstone

Edition status

Forthcoming after Course Edition 0.1. This chapter is part of the approved course architecture, but its lessons, exercises, and solutions have not yet been written. It is not represented as completed work.

Planned learning objectives

Integrate question design, relational data work, SQL, Python, statistics, visualization, time-aware modeling, responsible interpretation, and reproducibility.

Planned practical work

Deliver an evidence-backed analytical report, reproducible code, quality record, model documentation where applicable, and a short oral defense.

Existing project sources

The complete learning-analytics repository and all completed course modules.

26.1 What the roadmap does not imply

The presence of project scripts or saved artifacts does not mean the teaching modules above are complete. A future edition must still explain each idea from first principles, connect it to executable work, provide exercises and guided solutions, and verify every reported result. Until then, these entries remain explicitly forthcoming.

Part V

Reference Material

Guided Solutions

In plain language

These solutions model reasoning, not merely final answers. Try each exercise before reading its response. More than one wording or query can be correct when its grain, assumptions, and validation are explicit.

A.1 Course orientation

A.1.1 Solution to the orientation exercise

A defensible workflow begins with the question and source documentation because those establish what a row can mean. It then audits the source files, imports them into a raw layer without silent reinterpretation, and converts them into typed staging and core relations. Quality checks test row counts, keys, relationships, ranges, and missingness before analysis. SQL results are reconciled against known totals and described at their actual grain. Only then should a finding be communicated, together with its limits. This sequence is transferable because it relies on evidence and relational structure rather than on OULAD-specific commands.

A.2 Learning analytics and the problem

A.2.1 Solution to the classification check

“Withdrawn attempts account for 31.2% of recorded attempts” is descriptive. “Students with low activity are more likely to withdraw” is predictive or associational unless a causal design is supplied. “Sending a message reduces withdrawal” is causal and requires a comparison strategy that addresses what would have happened without the message. The important move is to classify the claim before choosing methods.

A.2.2 Solution to the question-design exercise

One bounded question is: “Among recorded student–module presentation attempts, how does the distribution of final results vary by module and presentation?” The unit is one attempt. The

population is the attempts present in OULAD, not all distance-learning students. The outcome is `final_result`. A later analysis must consider whether presentations differ in composition and timing. This question supports description; it does not identify why outcomes differ.

A.3 The OULAD data

A.3.1 Solution to the source-grain map

Table A.1: Suggested grains and candidate keys

Source	One row represents	Candidate key
<code>courses.csv</code>	One module presentation	<code>module + presentation</code>
<code>assessments.csv</code>	One assessment attached to a module presentation	<code>assessment_id</code>
<code>vle.csv</code>	One VLE activity type within a module presentation	<code>site_id + module + presentation</code>
<code>studentInfo.csv</code>	One student-module presentation attempt	<code>student_id + module + presentation</code>
<code>studentRegistration.csv</code>	One registration record for an attempt	<code>student_id + module + presentation</code>
<code>studentAssessment.csv</code>	One student's submission record for one assessment	<code>assessment_id + student_id</code>
<code>studentVle.csv</code>	One student's summarized interaction with one VLE site on one relative day	<code>site_id + student_id + module + presentation + date</code>

These are analytical candidate keys and should be tested, not trusted merely because they sound plausible.

A.3.2 Solution to the attempt trace

Begin with one attempt in `studentInfo.csv`. Match its registration record using student, module, and presentation. Find the presentation's assessments, then connect the student's submissions through `assessment_id` and `student_id`. Separately, match VLE activity through student, module, and presentation, with site and relative day adding detail. Keep assessment and VLE facts in separate aggregates until each has been reduced to attempt grain. Joining both detailed tables directly can multiply rows.

A.4 Relational foundations

A.4.1 Solution to the dangerous join

Suppose an attempt has four assessment submissions and 300 daily-site activity rows. A direct join between both detail tables can produce 1,200 rows for that attempt. Summing clicks after that join

would repeat each activity row four times. The repair is to aggregate assessments to one row per attempt, aggregate activity to one row per attempt, validate both intermediate keys, and only then join the two attempt-level relations.

A.4.2 Solution to the database-rules exercise

A useful rule set includes:

- a primary or unique key at the declared grain;
- foreign-key checks where the project intentionally enforces them;
- nonnegative click counts;
- accepted final-result values;
- explicit nullability for fields such as assessment due date; and
- row-count and orphan checks after every material transformation.

Database constraints prevent some invalid states. Quality queries remain necessary for conditions that are contextual, cross-table, or deliberately profiled instead of rejected.

A.5 PostgreSQL setup

A.5.1 Solution to the environment exercise

The environment record should identify Docker or local PostgreSQL, show the client and server versions, name the database, and include a successful `current_database()` query. For Docker, the Compose configuration is the authority for service, role, password, and database values. For a trusted local installation, `postgresql:///learning_analytics` can use the current operating-system user. The repository's default URL names a `learning_analytics` role, while the local database target may create only the database. That distinction should be diagnosed, not papered over.

A.6 Raw import

A.6.1 Solution to the reconciliation exercise

Create a three-column comparison for audited CSV, raw table, and typed core relation. For the source used in this project, all seven CSV counts should be accounted for. A mismatch is a stop condition until explained. First preserve the import log. Then confirm that the audited file and delimiter are the ones actually loaded. Finally, rerun only the smallest failing stage with stop-on-error enabled and inspect rejected or malformed records. Never adjust a target count simply to match an incomplete load.

A.6.2 Solution to the failed-cast exercise

Keep the raw text row unchanged. Query the candidate column with a narrow pattern or conversion guard to identify values that cannot be cast. Compare them with the data dictionary and neighboring

records. Then decide whether the value represents a documented missing marker, a source anomaly, or a parsing error. Implement the rule in staging, add a quality test, and rerun from a reproducible boundary. Editing the raw table would destroy evidence of the source condition.

A.7 Data validation

A.7.1 Solution to the validation specification

For `student_vle`, a compact specification includes:

- row count reconciles to the audited source;
- the declared composite key is unique;
- every module-presentation pair exists in courses;
- every site belongs to the corresponding VLE presentation;
- every student attempt exists at attempt grain;
- `sum_click` is nonnegative; and
- activity-day range and pre-start activity are profiled.

Pre-start records are a warning profile because they may be legitimate access, not automatically invalid rows.

A.7.2 Solution to the orphan-debugging exercise

Start with an anti-join that returns the orphaned child keys and a small sample, not only a count. Confirm that the join uses the full composite relationship; omitting presentation is a common mistake. Compare data types and whitespace normalization. Then inspect whether the parent source lacks the key or whether staging transformed it incorrectly. Fix the earliest reproducible cause and retain the anti-join as a regression test.

A.8 Introductory SQL

A.8.1 Solution to the inspection exercise

```

1 SELECT
2     student_id,
3     module,
4     presentation,
5     final_result
6 FROM core.student_info
7 WHERE final_result IN ('Fail', 'Withdrawn')
8 ORDER BY module, presentation, student_id
9 LIMIT 25;
```

This is an inspection query. The limit makes the display manageable but does not define the analytical population.

A.8.2 Solution to the outcome exercise

```
1 SELECT
2     module,
3     final_result,
4     COUNT(*) AS attempts
5 FROM core.student_info
6 GROUP BY module, final_result
7 HAVING COUNT(*) >= 100
8 ORDER BY module, attempts DESC;
```

The result is at module-by-outcome grain. The HAVING clause filters groups after aggregation; a WHERE COUNT(*) expression would be invalid.

A.8.3 Solution to the missingness exercise

```
1 SELECT
2     COUNT(*) AS attempts,
3     COUNT(imd_band) AS recorded_imd_band,
4     COUNT(*) FILTER (WHERE imd_band IS NULL) AS missing_imd_band,
5     ROUND(
6         100.0 * COUNT(*) FILTER (WHERE imd_band IS NULL) / COUNT(*),
7         1
8     ) AS missing_pct
9 FROM core.student_info;
```

COUNT(imd_band) ignores null values, while COUNT(*) counts rows. The expected missing count is 1,111 attempt records.

A.8.4 Solution to the repeat-appearance exercise

```
1 SELECT
2     student_id,
3     COUNT(*) AS attempt_count,
4     COUNT(DISTINCT module) AS module_count
5 FROM core.student_info
6 GROUP BY student_id
7 HAVING COUNT(*) > 1
8 ORDER BY attempt_count DESC, student_id
9 LIMIT 25;
```

This query identifies people with multiple attempt rows. It does not by itself tell whether the rows are retries, different modules, or another pattern; that requires examining module and presentation.

A.9 Cumulative lab

A.9.1 Solution to lab checkpoint 1

COUNT(*) counts the 32,593 rows in the attempt-level relation, so its unit is attempts. COUNT(DISTINCT student_id) counts the 28,785 unique student identifiers represented by those rows. The attempt

count is larger because some students appear in more than one module presentation. Neither measure is “more correct” without a question that names its unit.

A.9.2 Solution to lab checkpoint 2

For the Docker path, confirm the Compose service is running and use the role, password, port, and database declared there. Recreating the service from the project configuration may be appropriate when no persistent work would be lost. For a trusted local path, use the current local role with `postgresql:///learning_analytics`, or deliberately create and grant the named role. Import scripts cannot fix the error because authentication happens before SQL or CSV loading begins.

A.9.3 Solution to lab checkpoint 3

First save the failed command, full log, audited file checksum, and observed counts. Second, use a clean raw table and rerun the single `studentAssessment.csv` load with stop-on-error behavior, watching for delimiter, quoting, encoding, or newline errors. Third, compare the final file record with the loaded tail and verify that the source audit counted data rows the same way as PostgreSQL. Repair the loader or source acquisition step; do not invent the missing row.

A.9.4 Solution to lab checkpoint 4

One acceptable note is:

All 18 error-level tests passed, so the current build clears the project’s blocking quality gate. The suite also profiles 688,988 VLE activity rows before presentation day zero; these are retained, but later feature windows must state whether they include them. Eleven assessments have no due date, so deadline- relative features need a missingness rule. IMD band is missing for 1,111 attempts, so demographic summaries must report their denominator and missing category. These warnings document consequential source conditions rather than failed integrity tests.

A.9.5 Solution to lab checkpoint 5

The data contain 32,593 student–module presentation attempts contributed by 28,785 distinct students, so several students appear more than once. Pass is the most common recorded result (12,361 attempts), while 10,156 attempts (31.2%) are recorded as Withdrawn. These values describe attempts, not unique people, and should not be interpreted as causal explanations of learning outcomes. The blocking quality checks pass, but 688,988 activity rows occur before presentation day zero; any later engagement window must decide and document whether those rows are included. Assessment due-date and IMD-band missingness also constrain deadline-based or demographic comparisons. A useful next question is whether the outcome distribution differs across module presentations after first checking that each comparison uses consistent attempt-level denominators.

A.10 How to use these solutions

Compare the structure of your reasoning with the examples. A strong response usually names the unit, the population, the operation, the check, and the limit. Rewording is encouraged; unexamined copying is not.

Glossary

Table B.1: Course glossary

Term	Meaning in this course
Analytical grain	What one row represents in a relation or result. Also called the unit of observation.
Archive	One packaged file that contains other files, commonly in ZIP format.
Attempt	One student's participation in one module presentation. A student can have multiple attempts.
Audit	A recorded inspection of data or files against stated expectations.
Candidate key	A column or combination of columns expected to identify one row uniquely. It must be tested.
Cardinality	The number and pattern of relationships between rows, such as one-to-many.
Causal claim	A claim about what would change because of an intervention or exposure, not merely what is associated in observed data.
Code and script	Code is text containing computer instructions. A script is a file that runs a sequence of those instructions.
Command and terminal	A command is one instruction given to a computer. A terminal is the text-based application in which project commands are run.
Constraint	A database rule that restricts permitted values or relationships.
Core layer	The project's typed, named, analysis-ready relational layer.
CSV	Comma-separated values: a text format for tabular data.
Data and dataset	Data are recorded values or facts. A dataset is an organized collection of records.
Data science	The combined use of programming, databases, statistics, and subject knowledge to learn from data and communicate evidence.
Data dictionary	Documentation describing fields, values, tables, and their intended meanings.

Term	Meaning in this course
Database	An organized collection of data managed by a database system. In PostgreSQL, a database contains schemas, which contain relations and other objects.
Descriptive analysis	Analysis that summarizes what is recorded in a defined dataset and population.
Dimension	A descriptive attribute used to group or filter measures, such as module or presentation.
Evidence log	A record connecting a result to its query, unit, population, interpretation, and limitations.
Feature	A variable constructed for analysis or modeling. Its definition must include grain and information-availability timing.
Foreign key	A value or set of values that refers to a key in another relation.
Grain	See analytical grain.
Identifier	A value used to name or match a recorded entity. Whether it uniquely identifies a row depends on the table's grain.
IMD band	A binned index-of-multiple-deprivation field in OULAD. It contains missing values and should not be treated as a complete measure of an individual.
Information leakage	Use of data that would not be available at the intended decision time, or that otherwise reveals the target improperly.
Implementation	The working code and settings that perform a project, as distinct from documentation that explains it.
Input and output	An input is what a step receives; an output is what the step produces.
Join	An operation that combines rows from relations according to a match condition.
Lineage	The traceable path from original source data through transformations to a reported result.
Materialized view	A stored result of a query that must be refreshed when its inputs change.
Metric	A precisely defined value used to summarize something, including its unit and denominator where applicable.
Missing value	An absent or unavailable field value. Its meaning depends on source and context; it is not automatically an error or zero.
Module	An anonymized OULAD course unit.
Model	A mathematical rule used to summarize relationships or calculate predictions from input variables.
Normalization	Organizing relations to reduce redundancy and update anomalies.
Null	SQL's marker for a missing or unknown value. Comparisons use <code>IS NULL</code> , not equality to null.
Observation window	The time interval from which information is permitted when constructing an analysis or prediction.
OULAD	The Open University Learning Analytics Dataset used by this project.

Term	Meaning in this course
Orphan	A child row whose expected parent key cannot be found.
Path	The address of a file or directory, such as <code>sql/02_staging.sql</code> .
Pipeline	An ordered workflow in which the output of one step becomes input to another.
Population	The complete set of observations about which an analytical claim is made.
PostgreSQL	The open-source relational database system used by the project.
Presentation	A particular offering of a module, identified by an anonymized code.
Primary key	The database-enforced identifier for a row in a relation.
Prediction	An estimate of a later or unknown recorded outcome. It does not by itself explain why the outcome occurs.
Proportion	A part divided by a named whole. Multiplying a proportion by 100 produces a percentage.
Proxy	A recorded stand-in for a concept that is difficult to measure directly.
Quality gate	A set of blocking checks that must pass before dependent analysis proceeds.
Query	A request written in SQL that asks a database to return, combine, summarize, or modify data.
Raw layer	The source-preserving database layer, loaded primarily as text so source conditions remain inspectable.
Reconciliation	Comparison of totals or other invariants across source, raw, transformed, and reported stages.
Referential integrity	Consistency of relationships between child and parent keys.
Relation	A table-like set of rows and columns in the relational model.
Repository	The organized project folder and its tracked files.
Reproducibility	The ability to regenerate results from documented source, code, configuration, and environment.
Row and column	A row records one observation at the table's grain. A column stores one kind of value across rows.
Schema	A namespace inside a PostgreSQL database. This project uses schemas to separate raw, staging, core, and quality responsibilities.
Staging layer	The conversion layer where raw text is parsed, typed, normalized, and checked before publication to core relations.
Surrogate key	An identifier created by a system rather than derived from the natural domain attributes.
Unit of analysis	The entity about which a result is calculated or interpreted. It must agree with the query grain.
Unlogged table	A PostgreSQL table that avoids write-ahead logging for speed but has weaker crash-recovery guarantees.
Validation	Testing data against stated expectations while preserving evidence of any failure.

Term	Meaning in this course
Variable	A recorded characteristic that can differ across observations and is usually stored in a table column.
VLE	Virtual learning environment. OULAD records summarized interaction with VLE sites by student and relative day.
Warning profile	A nonblocking quality result that requires an analytical decision or interpretive note.
Window function	A SQL function that calculates across related rows without collapsing them in the way <code>GROUP BY</code> does.

References

Bibliography

Edgar F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970. doi: 10.1145/362384.362685.

Jakub Kuzilek, Martin Hlosta, and Zdenek Zdrahal. Open university learning analytics dataset. UCI Machine Learning Repository, 2015. URL <https://doi.org/10.24432/C5KK69>. Licensed under Creative Commons Attribution 4.0.

Jakub Kuzilek, Martin Hlosta, and Zdenek Zdrahal. Open university learning analytics dataset. *Scientific Data*, 4:170171, 2017. doi: 10.1038/sdata.2017.171. URL <https://doi.org/10.1038/sdata.2017.171>.

PostgreSQL Global Development Group. Postgresql 16 documentation. PostgreSQL documentation, 2026. URL <https://www.postgresql.org/docs/16/>.

Build and provenance note

The editable source for this edition is `docs/course/learning_analytics_course.tex`. The compiled PDF is `docs/course/generated/learning_analytics_course.pdf`. Build instructions are in `docs/course/README.md`.

The course was developed only as new documentation inside `docs/course/`. Existing analytical implementation and result files were read as sources and were not modified.